

Advanced Natural Language Processing

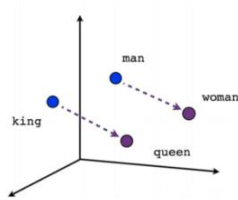
Deep Learning: Transformers Architecture

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2025-2026

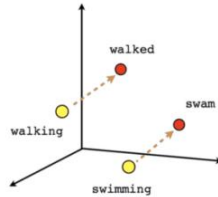


Maastricht University

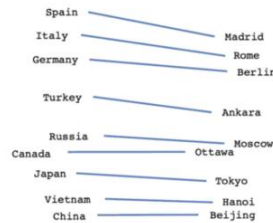
Transfer Learning through Word Embeddings



Male-Female



Verb tense



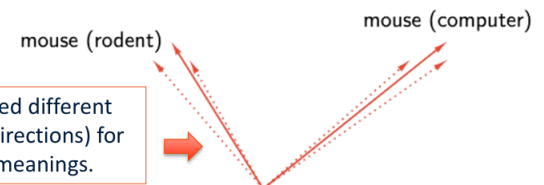
Country-Capital

Context Sensitive Embeddings

Sentences

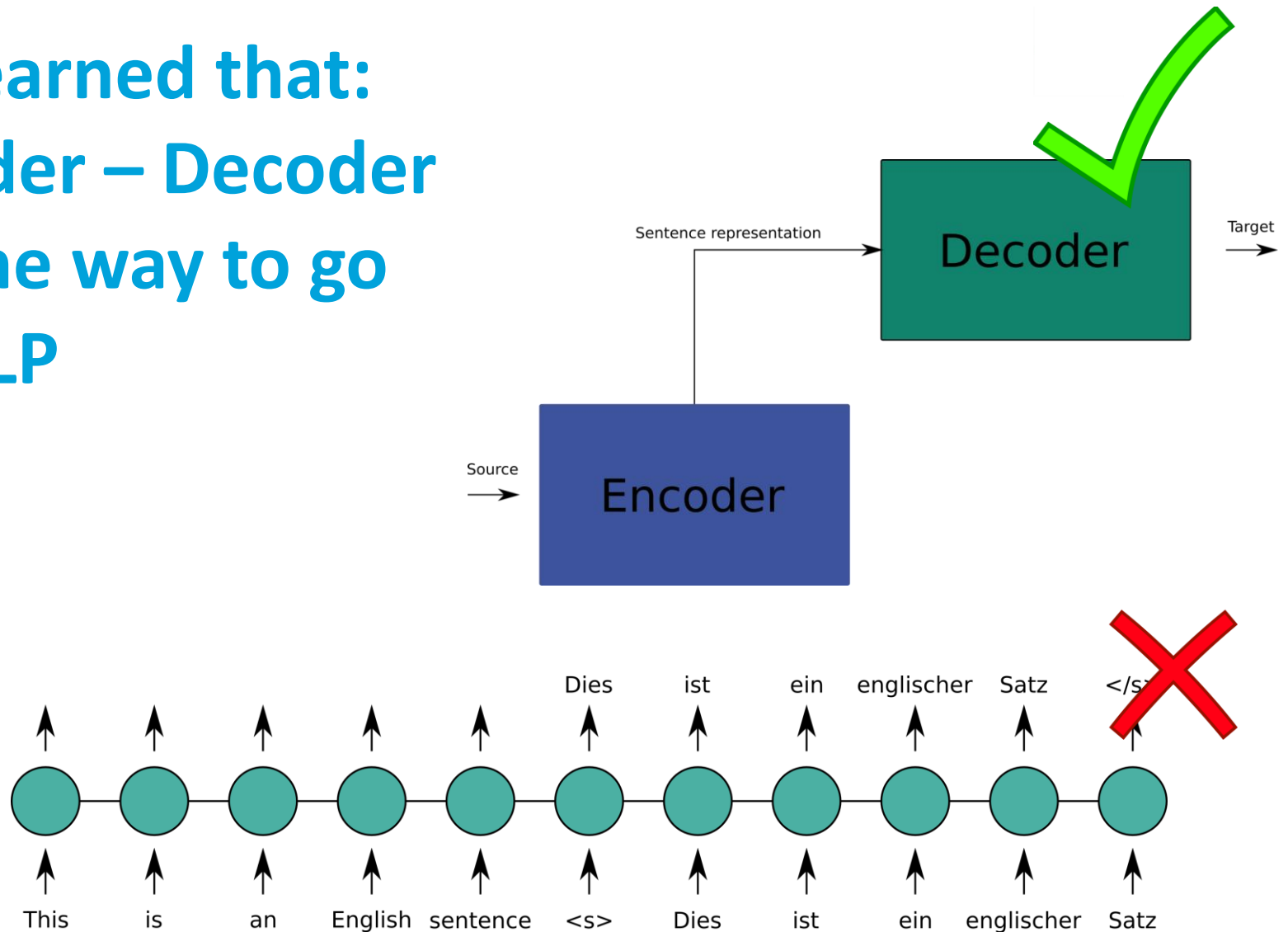
"bank"
"he eventually sold the shares back to the bank at a premium."
"the bank strongly resisted cutting interest rates."
"the bank will supply and buy back foreign currency."
"the bank is pressing us for repayment of the loan."
"the bank left its lending rates unchanged."
"the river flowed over the bank."
"tall, luxuriant plants grew along the river bank."
"his soldiers were arrayed along the river bank."
"wild flowers adorned the river bank."
"two fox cubs romped playfully on the river bank."
"the jewels were kept in a bank vault."
"you can stow your jewellery away in the bank."
"most of the money was in storage in bank vaults."
"the diamonds are shut away in a bank vault somewhere."
"thieves broke into the bank vault."
"can I bank on your support?"
"you can bank on him to hand you a reasonable bill for your services."
"don't bank on your friends to help you out of trouble."
"you can bank on me when you need money."
"i bank on your help."

The different meanings of the word "bank" will get very different vectors, depending on this context. So, the "financial bank" will be closer to other financial words than the "river bank". This is essential for good language understanding!



So, we need different vectors (directions) for different meanings.

We learned that:
Encoder – Decoder
are the way to go
for NLP

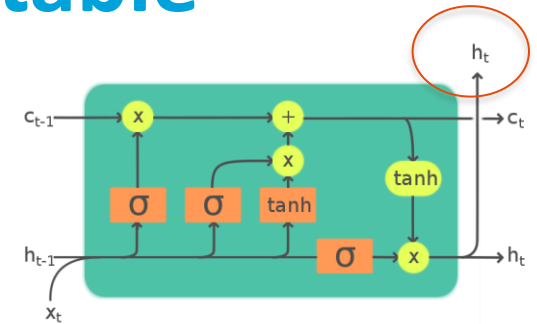


LSTM are too complex and unstable

Variables [edit]

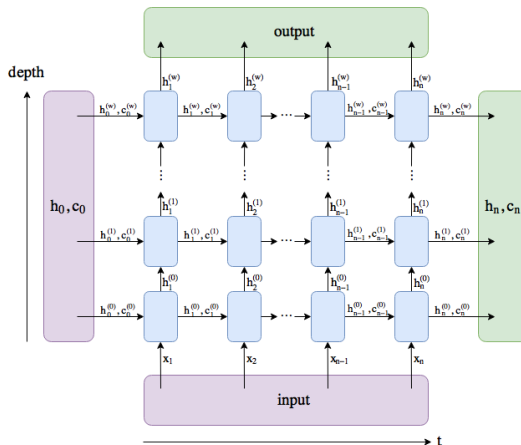
- $x_t \in \mathbb{R}^d$: input vector to the LSTM unit
- $f_t \in (0, 1)^h$: forget gate's activation vector
- $i_t \in (0, 1)^h$: input/update gate's activation vector
- $o_t \in (0, 1)^h$: output gate's activation vector
- $h_t \in (-1, 1)^h$: hidden state vector also known as output vector of the LSTM unit
- $\tilde{c}_t \in (-1, 1)^h$: cell input activation vector
- $c_t \in \mathbb{R}^h$: cell state vector
- $W \in \mathbb{R}^{h \times d}$, $U \in \mathbb{R}^{h \times h}$ and $b \in \mathbb{R}^h$: weight matrices and bias vector parameters which need to be learned during training

where the superscripts d and h refer to the number of input features and number of hidden units, respectively.



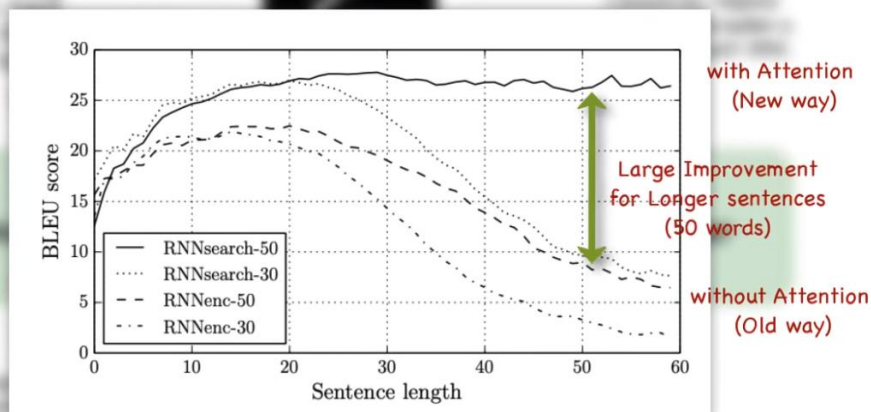
Legend:

Layer	ComponentwiseCopy	Concatenate
		$\updownarrow$



- $h(t)$ is hidden state vector. Dimensionality of $h(t)$ = *number of hidden unit layers*.
- d = *dimensionality of number features* (aka length of word embedding) of $x(t)$, typically 300-512
- Number of hidden layers (h) typically is between 3-6 in LSTM's, but can be larger.

We get too Low Performance on Longer Sentences without an attention mechanism



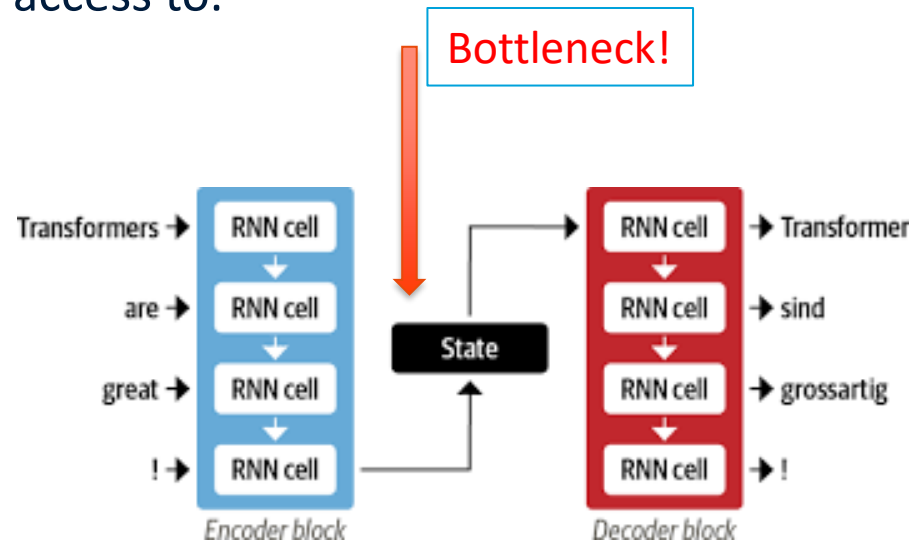
Source: Neural Machine Translation by jointly learning to align & translate (Bahdanau et al., 2016)

- Especially on longer sentences, the RNN mechanism to encode the entire sentence in the last hidden state does not work.
- We need something else: attention!

We cannot store attention in a small vector: too many relations.

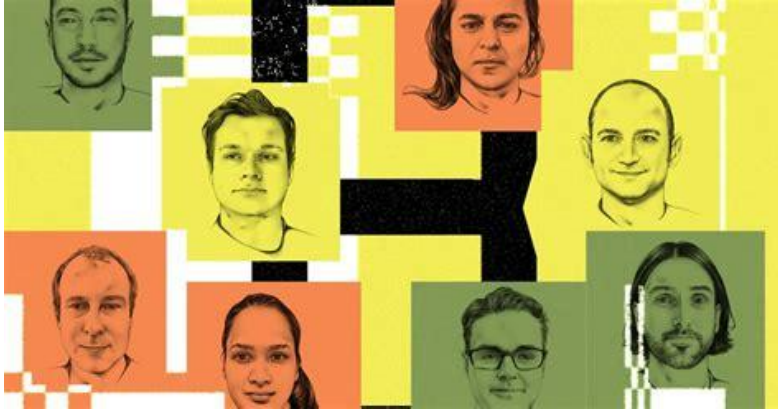
- LSTM's implement an encoder-decoder framework (aka sequence-to-sequence) architecture:
 - Input is converted into a hidden-state vector: encoder
 - Hidden-state vector is converted into output: decoder.
 - The last hidden-state contains all information of the entire string.
- Problem: final hidden state creates a bottleneck, because it has to contain all information the decoder has access to.

All attention is stored in a vector that is only 2-6 dimensions...



2017: “Attention is All you need”

WIRED



BY STEVEN LEVY THE BIG STORY MAR 28, 2024 6:00 AM

8 Google Employees Invented Modern AI. Here's the Inside Story

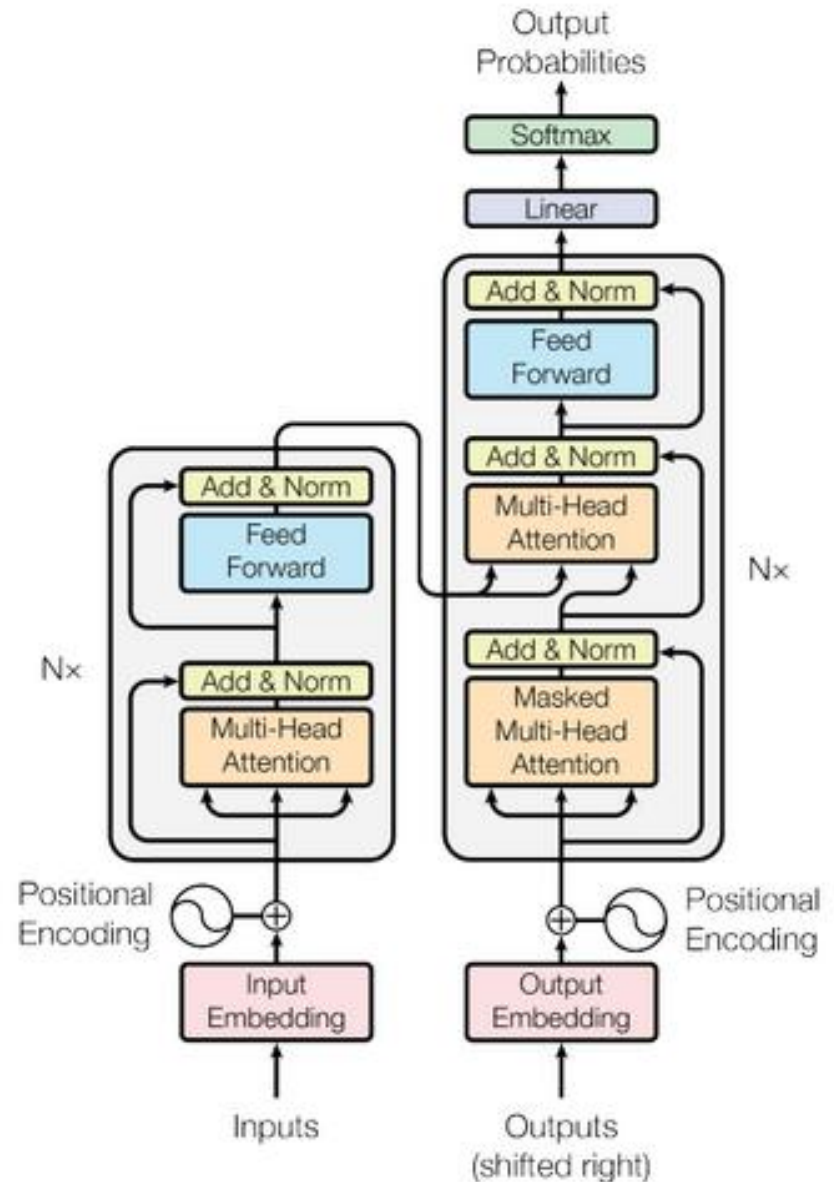
They met by chance, got hooked on an idea, and wrote the “Transformers” paper—the most consequential tech breakthrough in recent history.

The paper "Attention Is All You Need" introduced the Transformer architecture, which revolutionized natural language processing by using self-attention mechanisms to achieve state-of-the-art performance without relying on recurrent or convolutional networks.

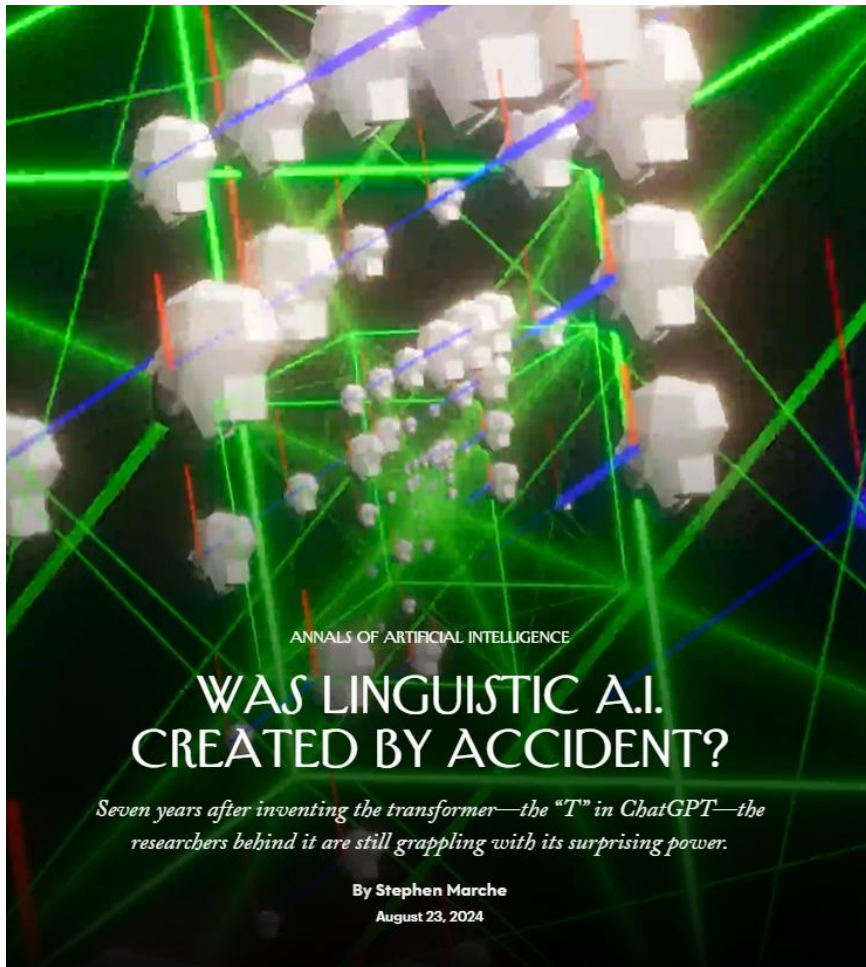
Attention was nothing more than relatively simple matrix multiplications. The chips used by Google did one thing really well: matrix multiplication. This allowed them to implement Self-attention at unprecedented speed.

Transformers

- Introducing the game-changer in the world of NLP: the Transformer model.
- Captured in a single, powerful image, this architecture redefined how we approach natural language processing and understanding. With its ingenious self-attention mechanism, the Transformer stepped beyond traditional RNNs and CNNs, enabling parallel processing and unprecedented efficiency.
- This model laid the groundwork for all the major breakthroughs that followed, from BERT to GPT, marking a new era where "Attention is All You Need" became the mantra that reshaped AI as we know it.



Chance or Plan?



- Many great inventions were discovered by chance.
- The authors of “Attention is all you Need” did understand their computational linguistics.
- Parallel computations allowed them to scale to a level never seen before. Bigger is better? Faster is better?
- Was it chance or a well executed plan?
- The true power of the transformer only became clear a few years later around 2018-2019.
- Nobody expected a development such as ChatGPT showed in 2022 to be possible by decoding-only transformers.
- Transformers are taking over AI, not only language, but also other applications such as image recognition.

Explaining Transformers

- 
1. The intuition behind Transformer Architecture and Multi-Headed Self-Attention (MH-SA)
 2. From Attention to Self-Attention and using Cross-Attention
 3. The mathematical definition of MH-SA
 4. The exact dimensions of the vectors (embeddings) and matrices of the calculation.
 5. All architectural components of the transformers.
 6. Cross Attention
 7. Detailed Example of a translation

Intuition Behind Transformers Architecture

- Word embeddings:
 - Use context-sensitive encodings so tokens with similar meaning will have similar encodings. Add positional embeddings to deal with sequential properties/distances and segment or context embeddings for homonyms.
- Encoder-Decoder:
 - encoder encodes linguistic relationships in a matrix for important and less important relations implementing alignment relations.
 - This matrix is used to drive the so-called cross-attention where a basic language model (implementing next-word prediction) can be directed to select not only linguistically correct words in a particular language but also select words that are relevant given the information in the encoder (so the correct translation-, -answer or -part of abstract).
- Multi-headed Self-Attention:
 - We will discuss this a bit further, but this is used to derive the correct values in the attention matrix which we discussed above.
- Feed-Forward layers:
 - This is required to train the model using back-propagation, so only the relevant information from one layer is forwarded to the next layer (a form of 1-dimensional convolution). See it as a convolution filter that can be trained or in other words, be derived automatically from the training data. Without such filters, every layer would do the same and there would not be any need for multiple layers. By filtering, each layer also gets a particular task which will result in a more stable training process.
- Multiple layers (Nx) of self-attention:
 - by repeating the process of calculating multi-headed self-attention and feed-forward multiple times, the system is able to derive obvious visual relations in the lower layers (punctuation, morphology) and more complex relations in the middle layers (syntax, basic semantics) and complex linguistic relation in the upper levels (resolving co-references & pronouns, long distance logical relations, etc.).
- SoftMax and Linear operation at the end of the decoder:
 - Linear operation allows for the addition of a bias vector (threshold) in the training process.
 - SoftMax converts the values coming out of the decoder into normalized (probability) values between [0-1] as with other deep-learning models.



Multi-Headed Self-Attention



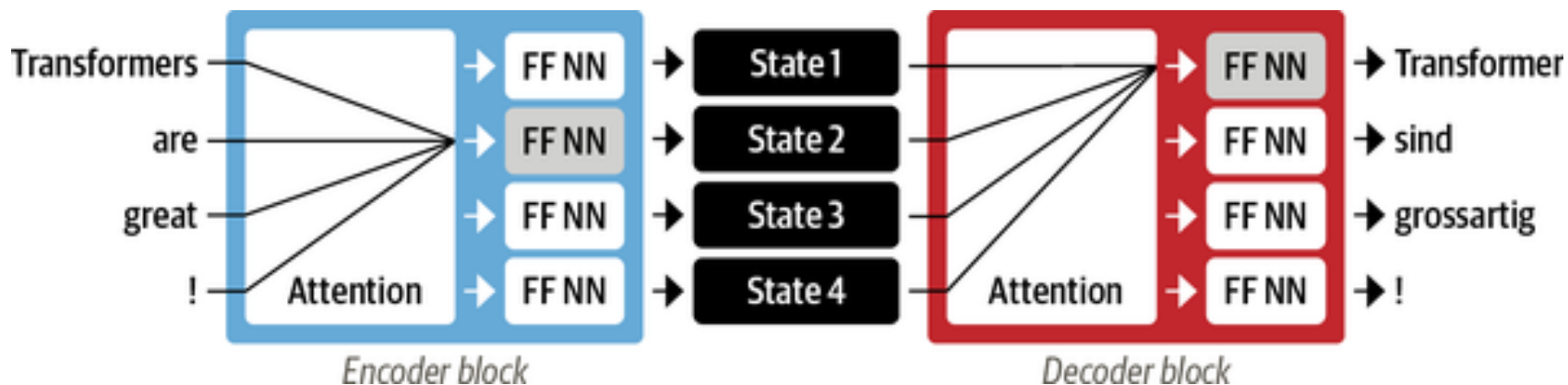
Maastricht University

Self-Attention

Kind of 1-dimensional
convolution

- Both the encoder and the decoder have their own self-attention mechanisms, whose outputs are fed to feed-forward neural networks (FF NNs) in transformers.
- This architecture can be trained much faster than recurrent models and paved the way for many of the recent breakthroughs in NLP.

How does this relate
to FSM and CFG?



Multi-Headed Self-Attention

- Self-Attention is a transformation to a latent space that allows us to understand (linguistic) relations better and to derive them automatically from exposure of the model to language.
- Multi-headed Self-Attention allows us to extract information from different representation subspaces, which would, otherwise, not be possible with a single attention head. This results in better disambiguation and alignment of linguistic structures.

Query, Key and Value in Self-Attention

- The key/value/query concept is analogous to retrieval systems.
- For example, when you search for videos on YouTube, the search engine will map your **query** (text in the search bar) against a set of **keys** (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (**values**).
- In self-attention, we map our **query** (sentence) with the **key** (same sentence), SoftMax (multi-class scaling), normalize and present best relations (**values**).

Values and weights of query, keys and values are set **during training** from exposure to natural language.

The Intuition Behind Multi-Headed Self-Attention – Part 1

- What are q , k and v exactly:
 - Now, q is the query (the search we need to do) and k tells us where to look, and v provides us with the values that will give us the desired content. Intuitively, the keys are the bridge between the queries (what we are looking for) and the values (what we will actually get).
- Compare all words to each other = $q \cdot k_i$ (before training):
 - Before training, q and k are vectors. Taking the dot product between q and k is like calculating a similarity measure (compare to cosine distance and dot product). If there is no direct match from q in k , then the dot product will give us the closest objects in k for q .
- From one token to the full sequence of tokens: QxK^T
 - Because we wish to calculate the MH-SA for a sequence of tokens and not for 1 token, we replace q by Q and k_i by K^T , where the rows contain the full embedding for each token in the sequence.
 - We also wish to train relevant relations for a specific linguistic task. So, the entire input sequence is a matrix $X \in \mathbb{R}^{batch \times tokens \times d_{model}}$ which we multiply by W^Q (set by training) to obtain matrix Q . We do the same for K^T .
 - So, by calculating QxK^T , we effectively calculate a dot-product on all tokens in a sequence.
 - As with the dot product, we need to normalize this with the square root of the length of the vector, in this case $\text{Sqrt}(d_k)$.
 - The result of QxK^T is a vector with non-normalized values, so we need to take the SoftMax to bring it back to the [0-1] range and make sure the sum of all values (per row) for the resolution of each elements in Q to an element in K is equal to 1.
 - Now we have a matrix representing a normalized similarity measures between all tokens in the input sequence.
- Multiply result with V for final weights:
 - Now we multiply the self-attention weights $\text{SoftMax}((QxK^T / \text{Sqrt}(d_k))$ from the previous operation with the values that we have in V . V consist of the original word embeddings for each token, multiplied by some training value, which results in the matrix V . By multiplying the self-attention weights $\text{SoftMax}((QxK^T / \text{Sqrt}(d_k))$ with these values, we obtain a (by self-attention) weighted version of all word embeddings (columns) for each token (rows) in the input sequence.

The Intuition Behind MH-SA (part 2)

- Multi-headed (64-dim sub spaces):
 - It is easier and faster to find latent relations in 8 smaller 64-dimensional spaces than in one large 512-dimensional space. Compare this with SVD or blocking. E.g.: complexity for MH-SA is $O(n^2)$ with n the number of tokens (remember, we compare n -token sequences to itself, so $n \times n$). $512 \times 512 = 262.144$, where $8 \times 64 \times 64$ is only 32.768.
 - TPU's and GPU's can do 64-dimensional calculation in one go.

As seen earlier:

- Feed-Forward layers:
 - This is required to train the model using back-propagation, so only the relevant information from one layer is forwarded to the next layer (a form of 1-dimensional convolution). See it as a convolution filter that can be trained or in other words, be derived automatically from the training data. Without such filters, every layer would do the same and there would not be any need for multiple layers. By filtering, each layer also gets a particular task which will result in a more stable training process.
- Multiple layers ($N \times$) of self-attention:
 - by repeating the process of calculating multi-headed self-attention and feed-forward multiple times, the system is able to derive obvious visual relations in the lower layers (punctuation, morphology) and more complex relations in the middle layers (syntax, basic semantics) and complex linguistic relation in the upper levels (resolving co-references & pronouns, long distance logical relations, etc.).

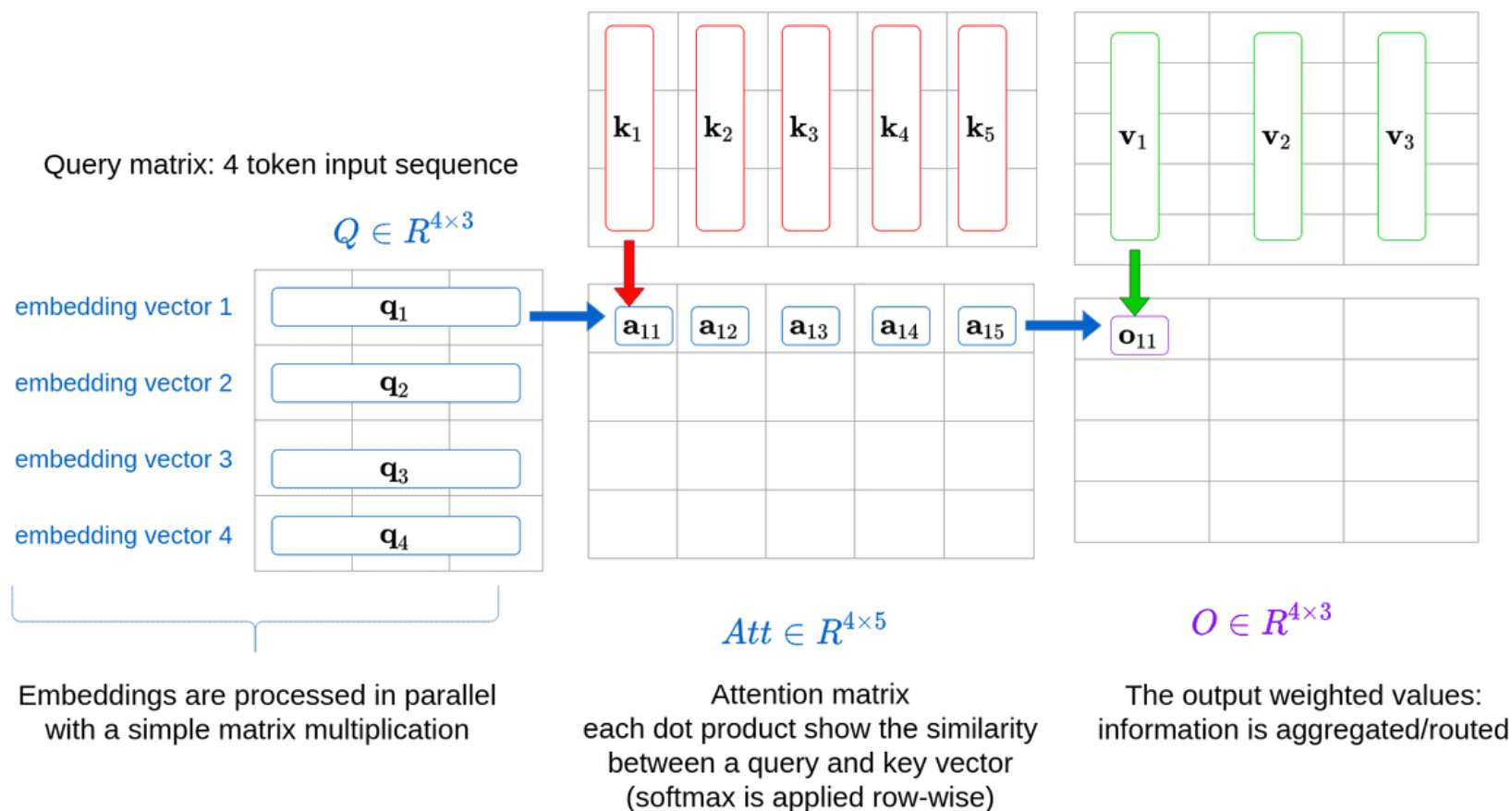


Intuition MH-SA encoding: Visually Explained

Keys, Values: 5 token sequence to associate with the input queries

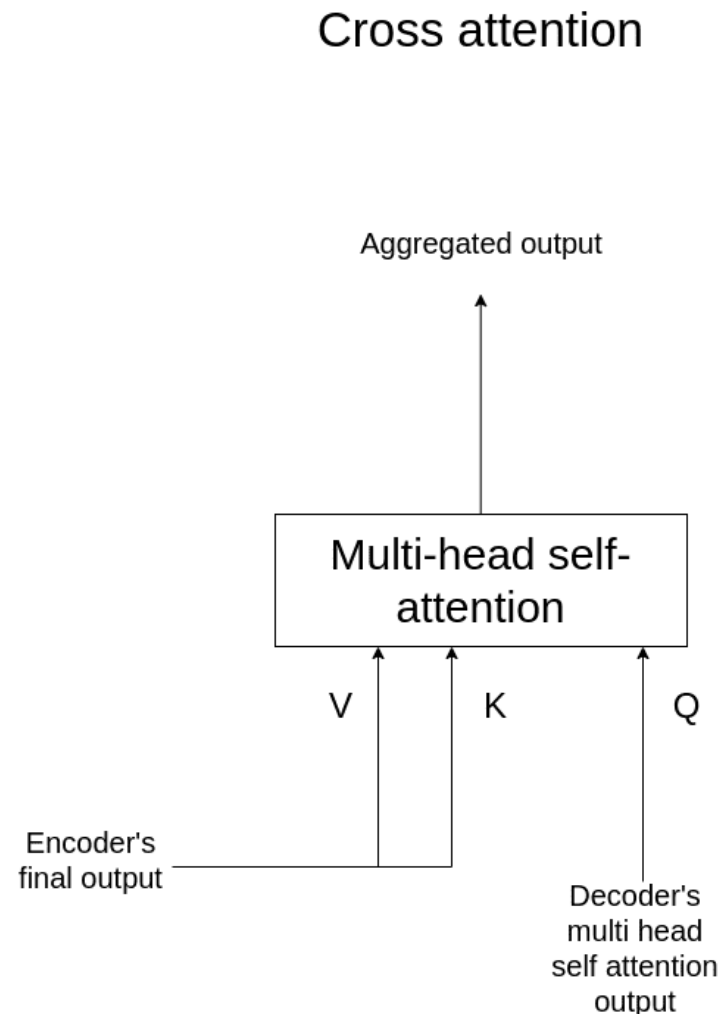
$$K^T \in R^{3 \times 5}$$

$$V \in R^{5 \times 3}$$

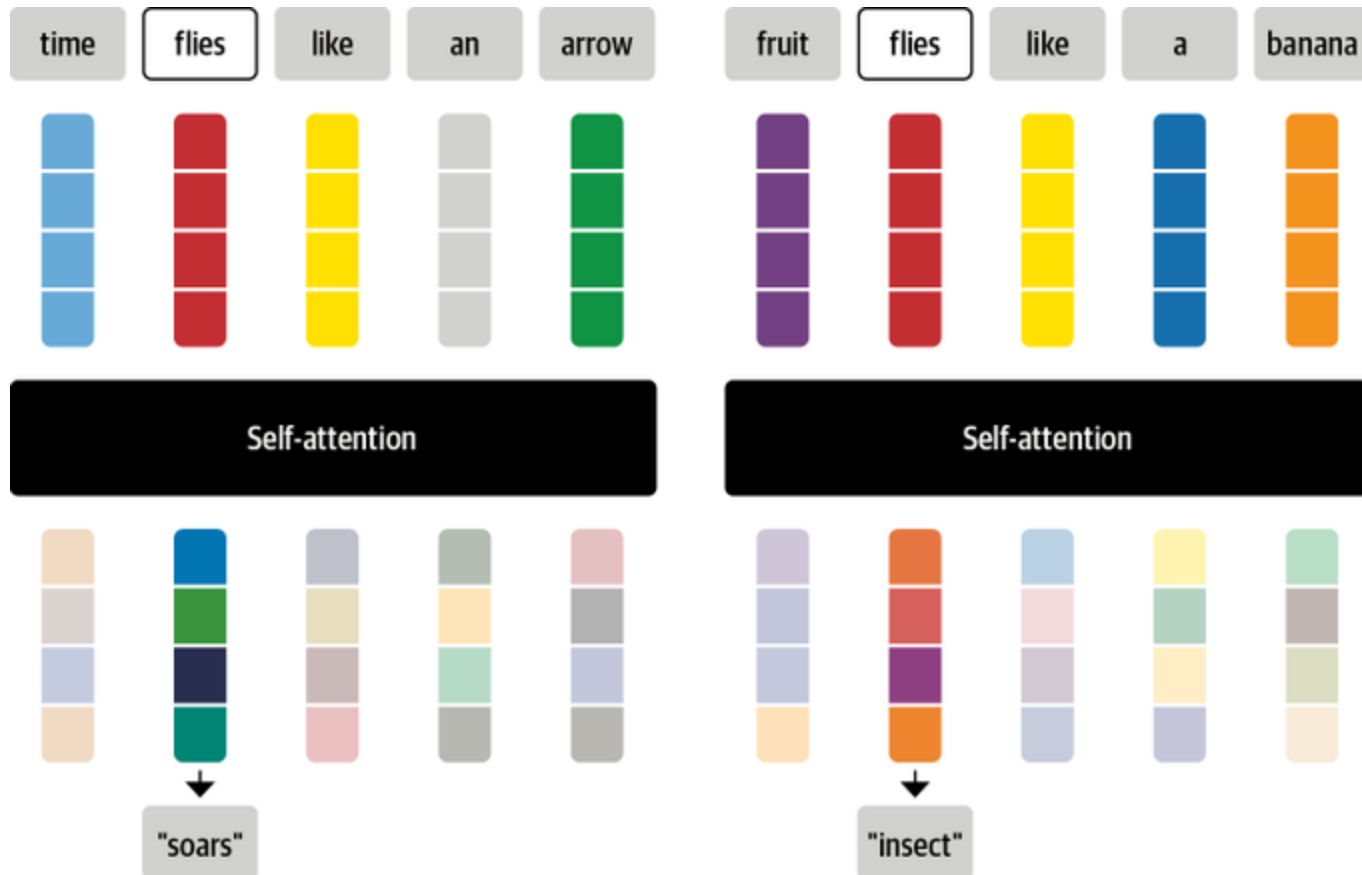


The Intuition behind Cross-Attention

- In the cross-attention (where the encoder information is used for the decoding), the language models predict a number of potentially relevant words based on next-word prediction: Q.
- In order to select the correct one for the linguistic task, we need to lookup K and V from the encoder information: the final MH-SA output.



Disambiguating with Self-Attention



Let's take a closer look into the details of MH-SA



Maastricht University

Multi-Headed Attention

- Q = queries (for each word in a sentence)
- K = keys (all other words in same sentence)
- V = values (again all words in same sentence)

Compute Self-attention by:

1. Compare all words to each other = $q.k_i$ before training and after training with weights: QxK^T
2. Take SoftMax from result $q.k_i$ (before training) and QxK^T (after training) to bring back to $[0,1]$.
3. Normalize SoftMax(QxK^T) (to scale back after matrix multiplications: divide by d_k)
4. Multiply result with V for final weights. Result is matrix of # words x # words with all attention weight from both complex word relations (pronoun, references, etc.) and word alignment.
5. Z = Self-Attention (one layer, not parallel Or multi-headed yet)

- Dimensionality Q, K, V = #words x d_k
- Values of Q,K, and V are set in by training NN using all context available
- Number of heads = h
- (Dimensionality word embedding) / # heads = d_k
- d_k = dimensionality per head

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \text{3x3 grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{3x3 grid} \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} \text{V} \\ \text{3x3 grid} \end{matrix} = \begin{matrix} \text{Z} \\ \text{3x3 grid} \end{matrix}$$

From Attention to Self-Attention to Multi-Headed Self-Attention

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^4 \exp(e_{ik})}$$

where $e_{ij} = \text{fc}(s_{i-1}, h_j)$

Dot product $q \cdot k_i$ attention without training (SoftMax on dot-product from 2 word embedding vectors)

Matrix multiplication after training resulting in weights for q and k

$$A(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} v_i$$

Becomes: $A(Q, K, V) = \text{softmax}(QK^T)V$

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

Dimensionality of word embeddings (normalization)

X = input vector,
rows = 512,
column = tokens

$$X \times W^Q = Q$$

$$X \times W^K = K$$

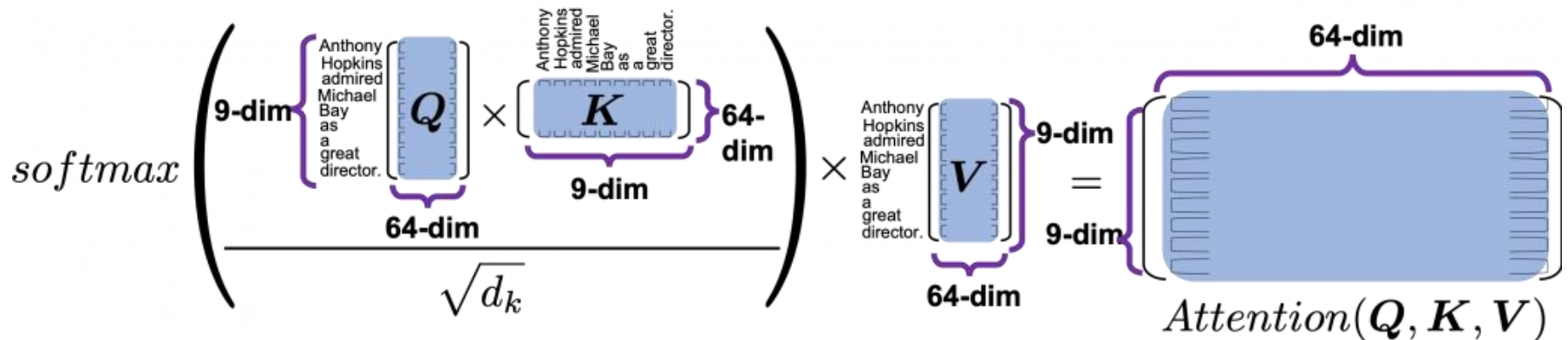
$$X \times W^V = V$$

Set by training

$$\begin{aligned} \text{head} &= \text{Attention}(QW^Q, KW^K, VW^V) \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Calculating Attention(Q,K,V) = SoftMax(QK^T)V

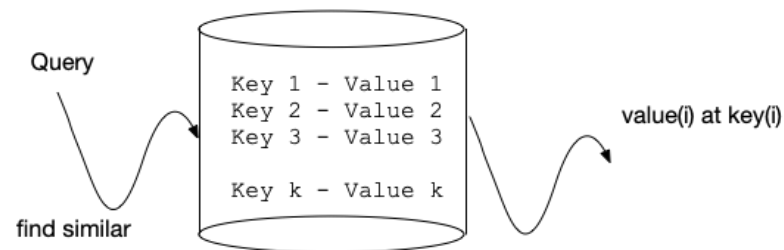


As you can see in the figure above, calculating Attention(Q,K, V) is virtually just multiplying three matrices with the same size (Only K is transposed though). The resulting 9x64 matrix is the output of the head.

64 probably fits best on GPU and TPU hardware for matrix multiplications. However, I did not find any references for this hypothesis.

(Self) Attention as Queries, Keys, and Values

- When building a language model, we're interested in the similarity measure of a word with respect to every other word in the source sequence and this can be formulated as the retrieval (from a database) with a query q of a value $v(i)$ available at the key, $k(i)$.
- Dimensionality of one head is $d_k = 64$ in our example
- Q , K and V are matrices including ALL vectors for parallel calculations and to make sure we compare everything to everything.



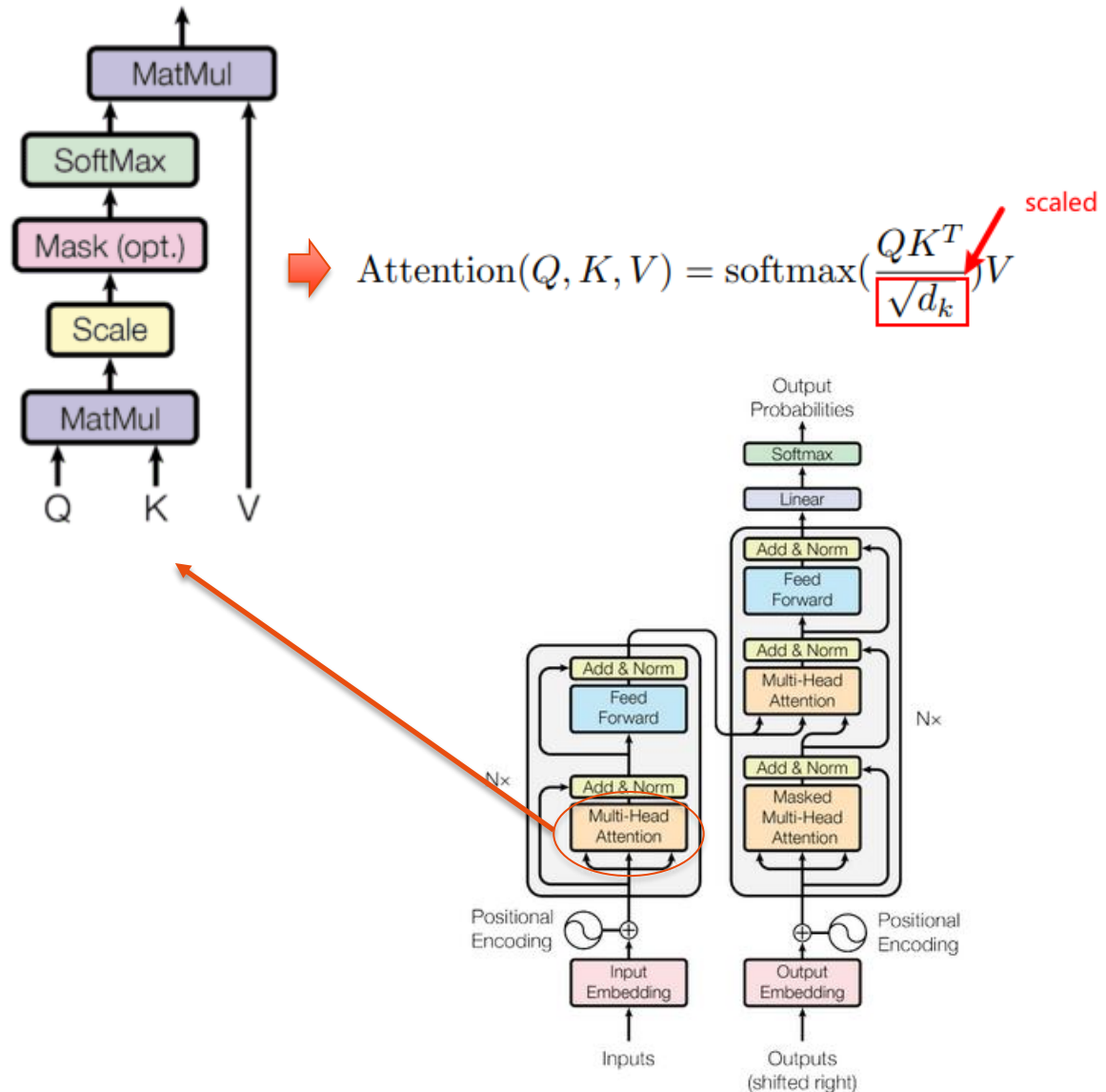
Self-Attention can be defined as a query that performs the best “match” against a set of keys that are then used to retrieve a value associated with the key.

$$\text{softmax} \left(\frac{\begin{matrix} \text{Anthony} \\ \text{Hopkins} \\ \text{admired} \\ \text{Michael} \\ \text{Bay} \\ \text{as} \\ \text{a} \\ \text{great} \\ \text{director.} \end{matrix} \begin{matrix} Q \end{matrix} \times \begin{matrix} \text{Anthony} \\ \text{Hopkins} \\ \text{admired} \\ \text{Michael} \\ \text{Bay} \\ \text{as} \\ \text{a} \\ \text{great} \\ \text{director.} \end{matrix} \begin{matrix} K \end{matrix}}{\sqrt{d_k}} \right) = \begin{matrix} \text{Importantly,} \\ \text{the sum of} \\ \text{each row is 1.} \end{matrix}$$

The diagram shows the calculation of the attention weights. On the left, a matrix Q (represented by a blue rectangle) is multiplied by a matrix K (represented by a blue rectangle). The result is divided by $\sqrt{d_k}$. The result is then passed through a softmax function. The result is a matrix of circles, where each row represents a word in the query and each column represents a word in the key. The circles are colored blue, indicating the attention weights. An arrow points to the matrix with the text 'Importantly, the sum of each row is 1.'

Scaled-Dot-Product Attention

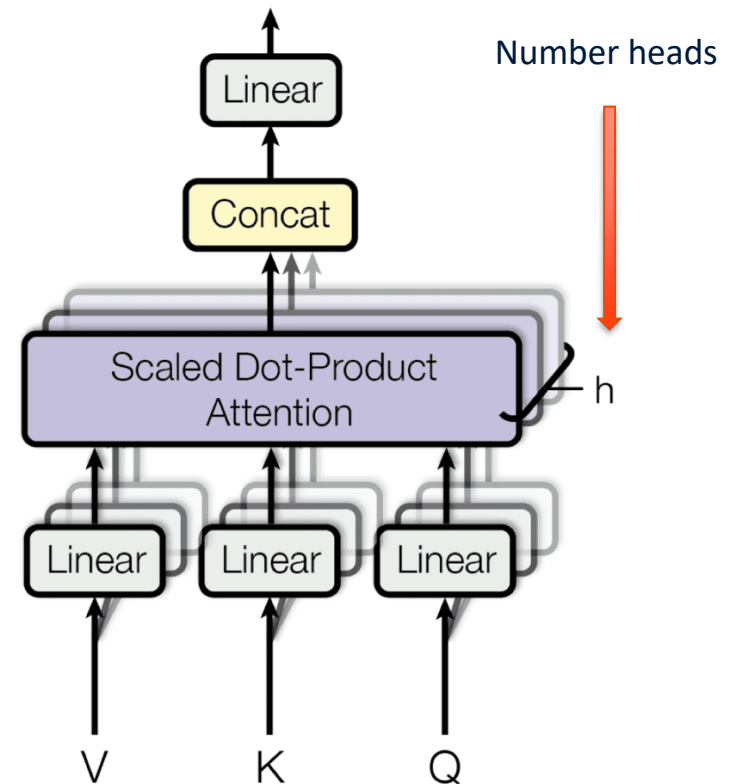
- The Transformer implements a scaled dot-product self-attention (only normalized with d_k) we defined earlier.
- Weights from matrices Q, K and V are set by training.



Multi-Headed Self-Attention

The transformer also implement a multi-headed self-attention mechanism:

- Multi-head attention mechanism linearly projects the queries, keys and values times, each time using a different learned projection. The single attention mechanism is then applied to each of these projections in parallel, to produce outputs, which in turn are concatenated and projected again to produce a final result: so, we compare everything with everything!
- The idea behind multi-head attention is to allow the attention function to extract information from different representation subspaces, which would, otherwise, not be possible with a single attention head.
- W^Q , W^K , and W^V denoting projection matrices that are used in generating different subspace representations of the query, key and value matrices. These start random, but are adjusted during training, as is W^O , the projection matrix for the multi-head output.



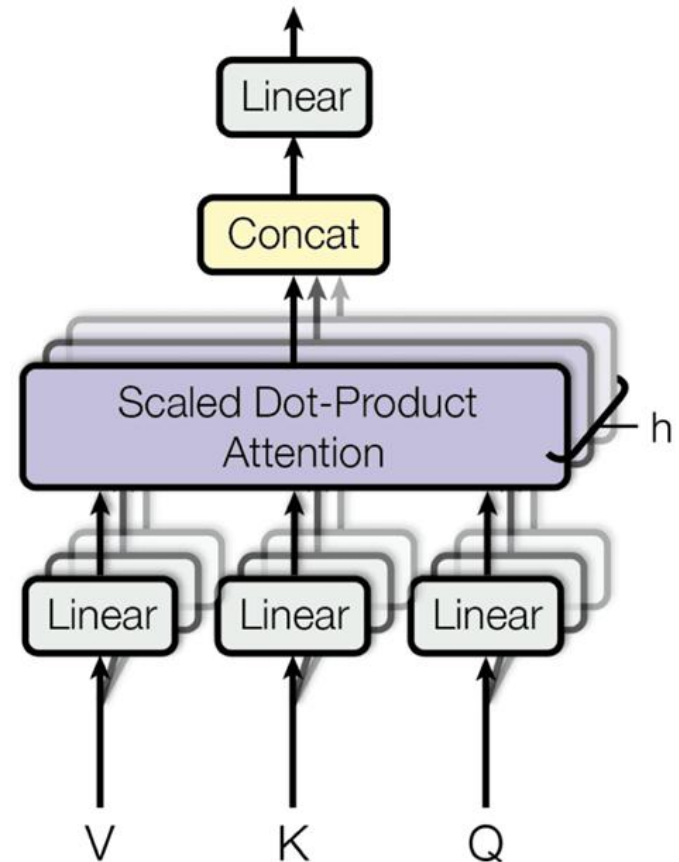
$$\text{head} = \text{Attention}(QW^Q, KW^K, VW^V)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Dimensionalities

- Word embeddings have dimensionality of 512 (in transformers).
- Internal embeddings BERT have dimensionality of 768.
- Typically:
 - Transformers 8 heads = h
 - BERT: 12 heads in self-attention = h
- V, K and Q have dimensionality $512/8$ or $768/12 = 64$ by “number of words in context” (max 12) or $512 / 8 = 64$ (in case of 8 heads).
- Dimension Concat = $12 \times 64 = 768$ (BERT) or 8×64 (Transformers)



MH-SA Dimensionalities

Symbol	Shape	Meaning
Q	$n \times d_k$	Query vectors (one per token)
K	$n \times d_k$	Key vectors (one per token)
V	$n \times d_v$	Value vectors (one per token)
QK^T	$n \times n$	Pairwise attention scores between tokens
$\text{softmax}(QK^T / \sqrt{d_k})$	$n \times n$	Attention weights (row-normalized)
Output: attention $\times V$	$n \times d_v$	Weighted combination of values

Step-by-step:

1. QK^T

$$(n \times d_k) \cdot (d_k \times n) \rightarrow (n \times n)$$

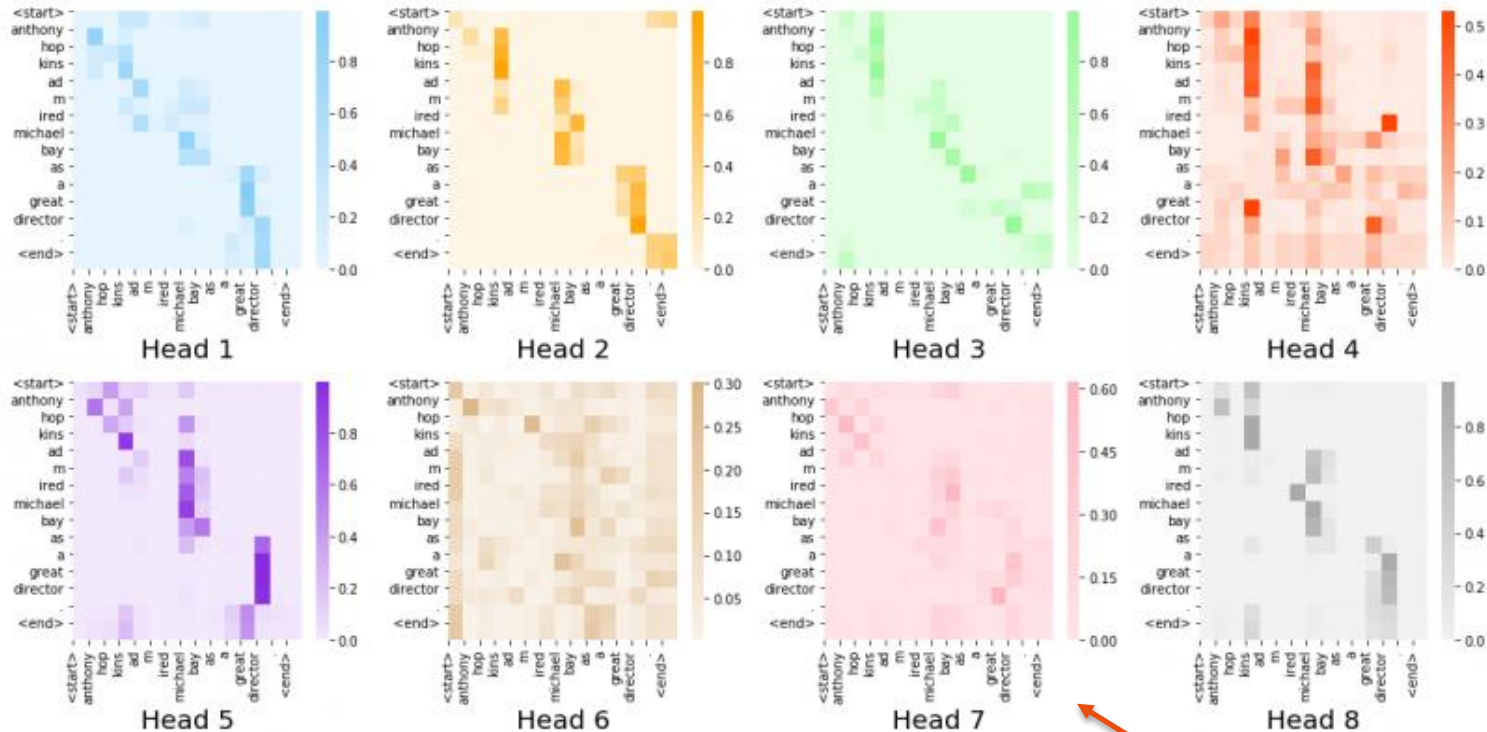
2. Softmax normalization stays $(n \times n)$.

3. Multiply with Value matrix V :

$$(n \times n) \cdot (n \times d_v) \rightarrow (n \times d_v)$$

The Different Heat-Maps of Self-Attention

The heat maps of self attentions of "Anthony Hopkins admired Michael Bay as a great director." in encoder_layer3_block



9 tokens => 8 heads. From 512 per embedding / 8 to 64 per head.

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

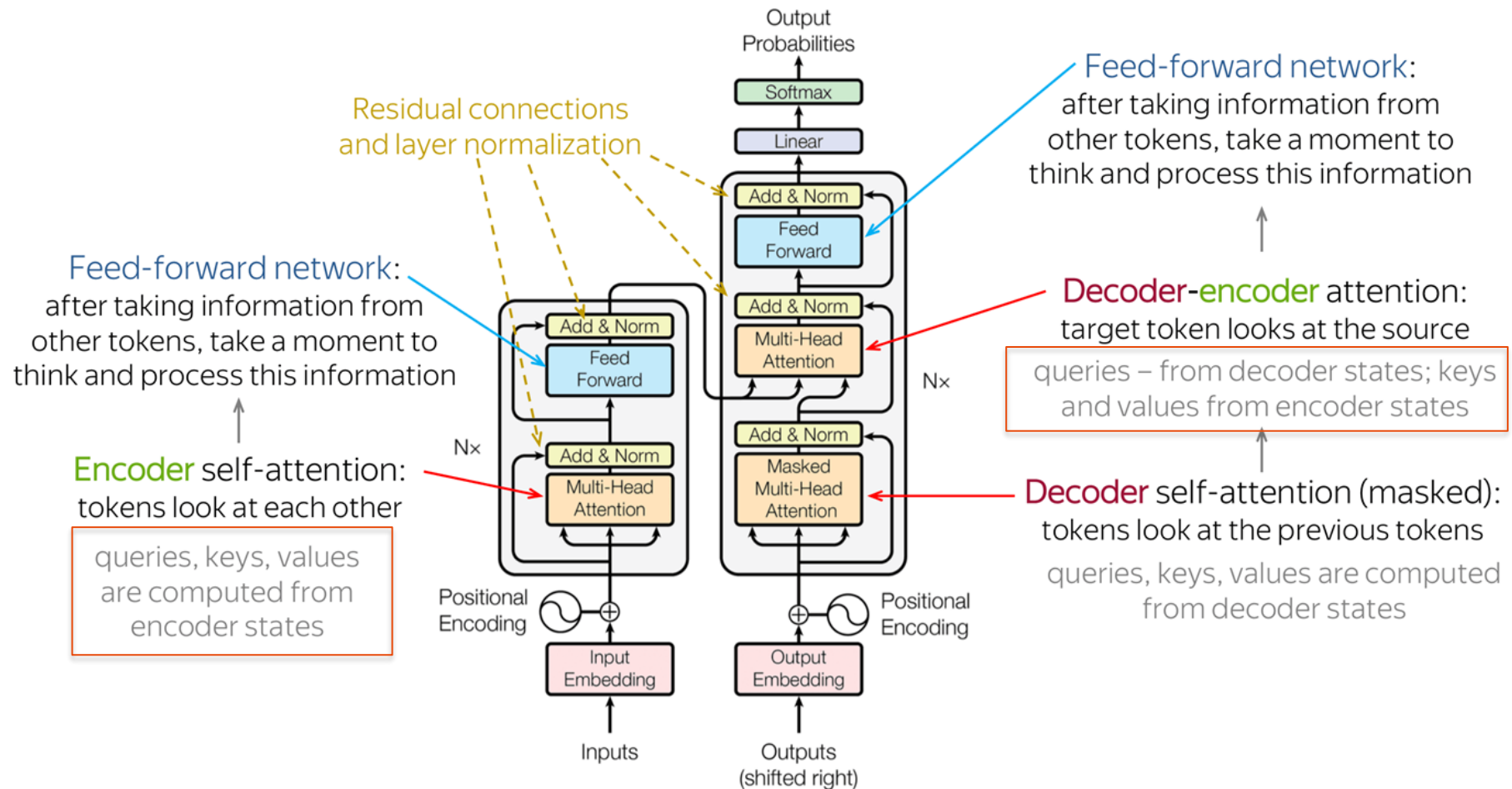
Detailed look at other parts of the Transformer Architecture

Feed Forward Layers



Maastricht University

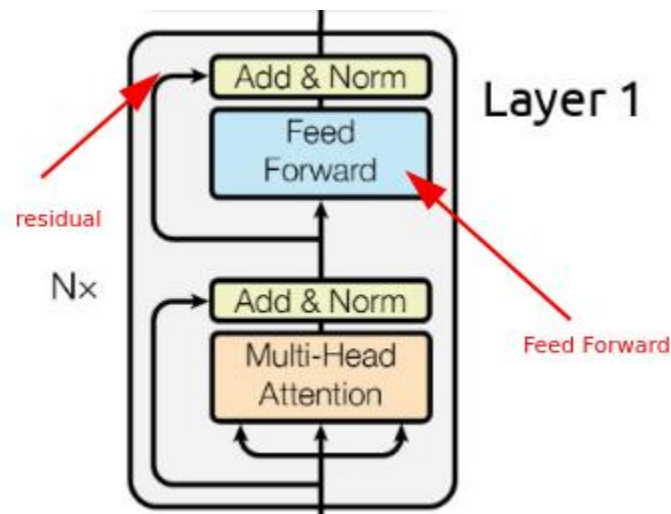
Let's Take a closer look at Transformers



What does the architecture of the Feed Forward Layers look like?

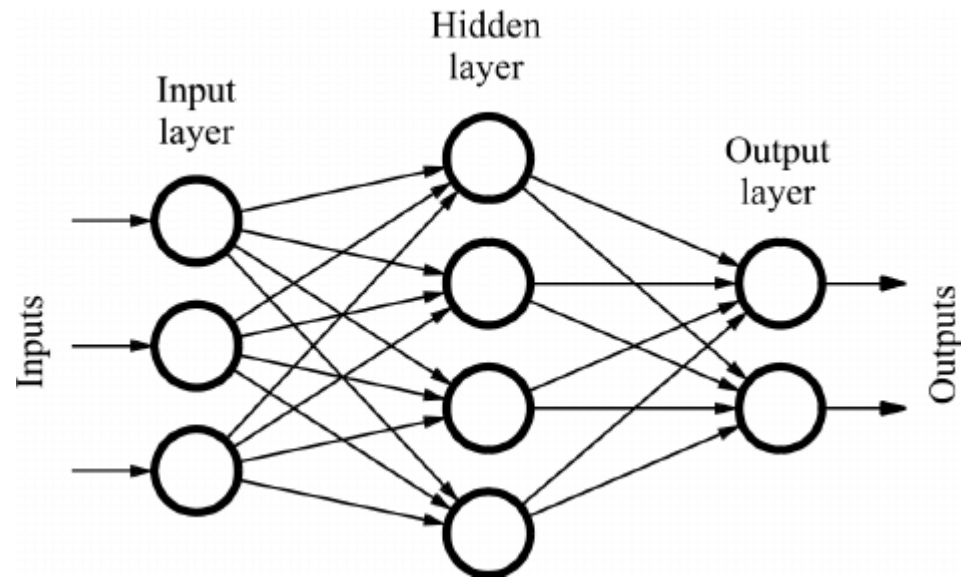
Are the trained?

- It is a position-wise transformation that consists of linear transformation, ReLU, and another linear transformation.
- The feed-forward layer has weights which are trained. The exact same matrix is applied to each respective token position.

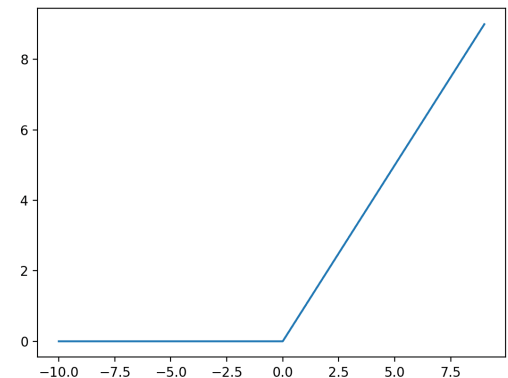


$$\text{ffLayer} = \sum_i \text{relu}(q_i k_i^T + b_i) v_i + c$$

Feed Forward



- This is how feed forward looks like normally. More hidden neurons in hidden layer than in input and output.
- Transformers only implement the hidden layer.
- Often use ReLU activation function.



Feed Forward

Let's take the common translation task which transformers can be used for as an example: If you would like to translate English to German one example of your training data could be

("the cat is black", "die Katze ist schwarz").

In this case your target is simply the German sentence "*die Katze ist schwarz*" (represented by embeddings incl. positional information). This is what you calculate your loss on, run backprop on, and derive the gradients as well as weight updates from.

Accordingly, you can think of the light blue feed forward layers of a transformer



as a hidden layer in regular feed forward network. Just as for a regular hidden layer its parameters are updated by running backprop base

Why Feed-Forward after Self-Attention?

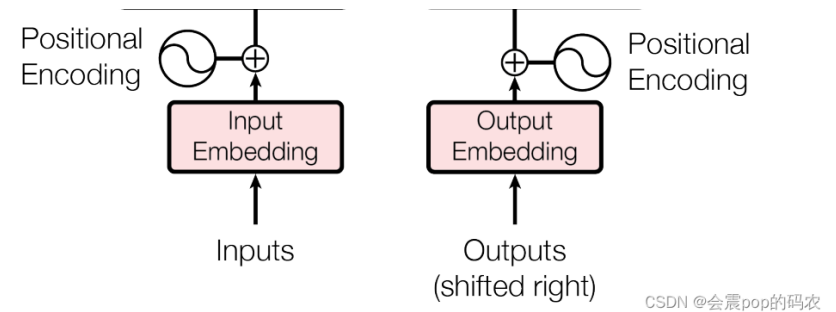
- If there is no feed-forward layer, self-attention is simply performing re-averaging of value vectors.
- If there is no feed forward, all attention layers will act the same, so no added value for higher up layers.
- Adding feed forward makes each attention layer behave like a separate small model that can be trained (get parameters). Then, in the whole process of stacked self-attention, each layer gets a different “filter” by training.

Purpose of a feed forward layer is to have something to train

Transformer Input

Input Embeddings
Output Embeddings
Positional Embeddings

Input Embeddings

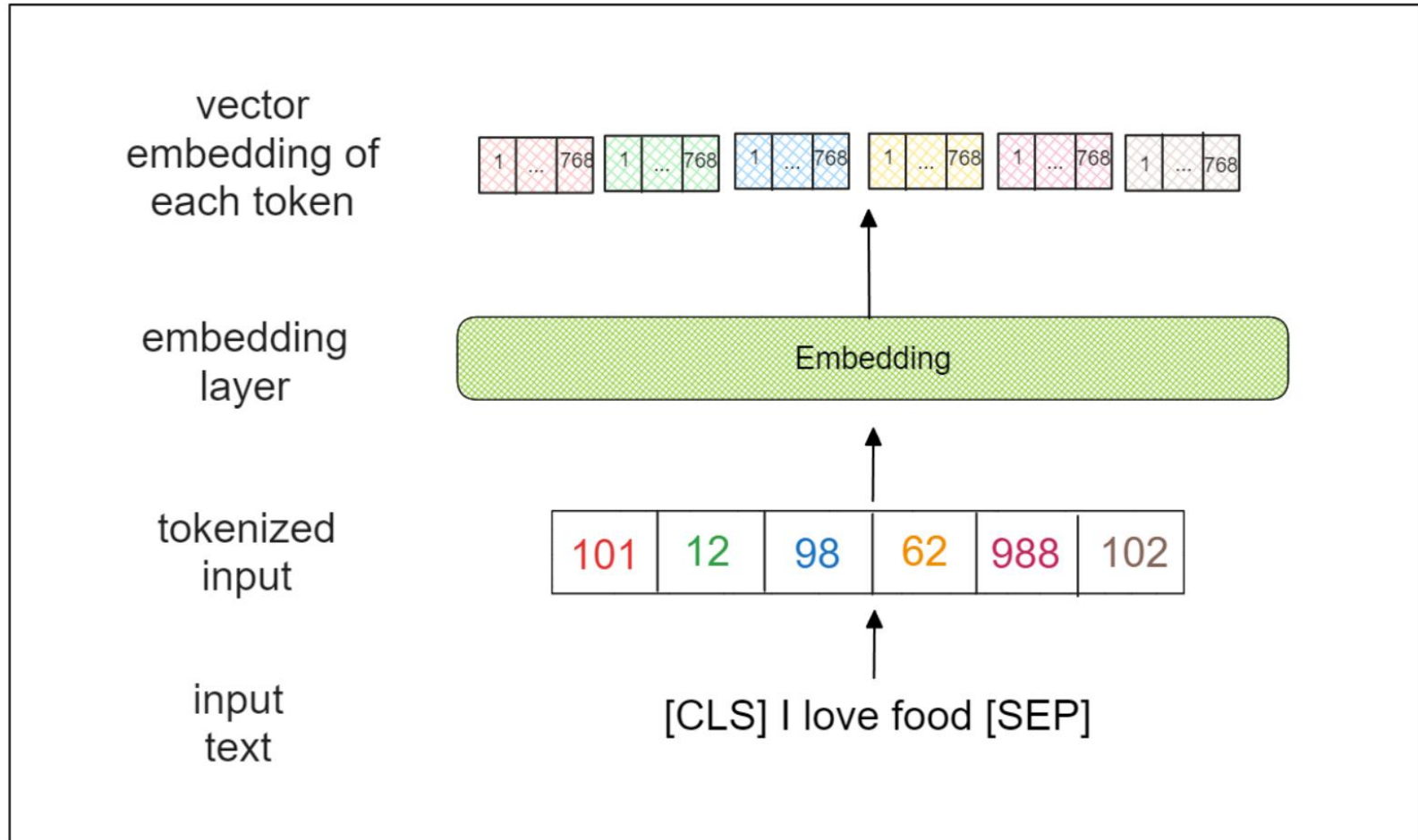


Transformer **input embeddings** are the numerical vectors the model uses as its starting point to process text. They combine two pieces of information:

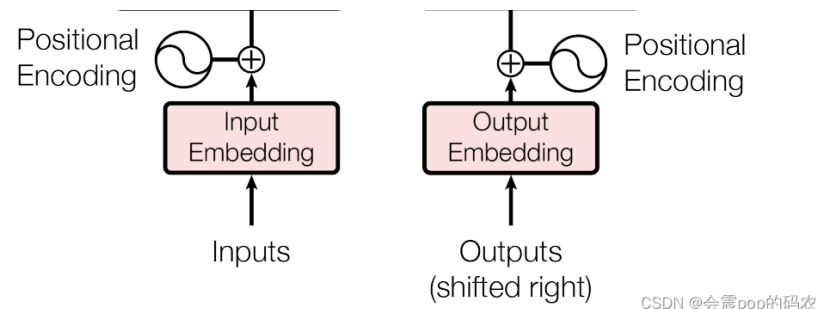
- **Token embeddings** – each input token (word, subword, or character) is mapped to a dense vector of size d_{model} via a learned lookup table.
- **Positional encodings** – since self-attention has no notion of order, a position signal is added to the token embedding (either fixed sinusoidal or learned) to tell the model where each token is in the sequence.
- $$\text{Input embedding for token } i = \text{WordEmbed}(t_i) + \text{PositionalEncoding}(i)$$

This d_{model} -dimensional vector is then fed into the first Transformer layer.

Token Embeddings



Output Embeddings



Decoder:

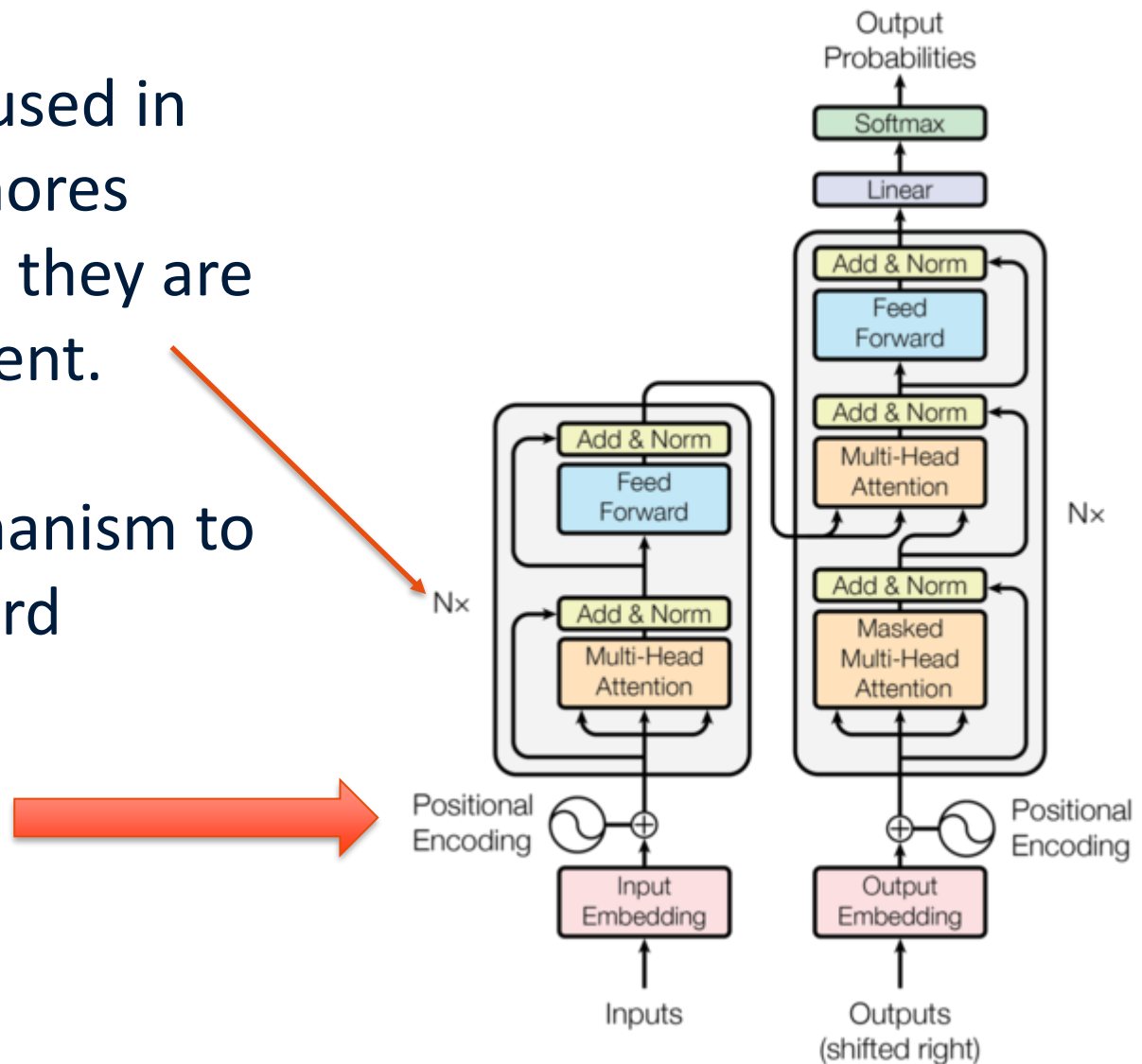
- **Output Embedding** (sometimes called *target embeddings* or *decoder input embeddings*) = token embeddings of the **already generated target tokens**, but **shifted right**.
- “Shifted right” means the decoder sees $\langle s \rangle$ (start token), then token y_1 , $y_2 \dots$, to predict $y_2, y_3, \dots$.
- This ensures the model only has access to **past tokens**, never the one it is currently predicting.

The **Output Embedding block at the bottom of the decoder** is the embedding of the **text that has already been generated**:

- during training: teacher-forced ground-truth tokens;
- during inference: the model’s own past outputs).

Positional Encoding in Transformers

- Representation used in Transformers ignores word position as they are no longer recurrent.
- We need a mechanism to keep track of word positions



Why Positional Embeddings?

Without positional embeddings we cannot distinguish:

“Jane hits John” from “John hits Jane”

Disambiguation is easier using positional information. Consider the following sentence:

“if 'are' comes after 'they', then 'playing' is more likely than 'play’”

This is essential FUNCTIONALITY for all layers N_x of the encoding and decoding of the transformer model.

First Idea

- The first idea: to assign a number to each token in the sequence within the $[0, 1]$ range in which 0 means the first token and 1 is the last token and use a time-step delta of $1/\text{\#tokens}$
- Does not work well: you cannot figure out how many words are present within a specific range: in other words, “time-step delta doesn’t have consistent meaning across different sentences”. See example below:



“I hit the ball very hard” versus “



One day, there was a ball, which was red and large, that I was capable of hitting very hard”

See also: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/ for more explanations and examples.

Another Idea

- Another idea is to assign a number to each time-step linearly. That is, the first token is given a “1”, the second token is given a “2”, and so on.
- The problem with this approach is that:
 - Values could get quite large,
 - We can run into sentences longer than the ones fitting in the window
 - We may not see any sample with one specific length for specific tokens which would hurt generalization.



“I hit the ball very hard” versus “



One day, there was a ball, which was red and large, that I was capable of hitting very hard”

Requirements Positional Encoding

Ideally, the following criteria should be satisfied:

- It should output a unique encoding for each token (e.g. time-step) position in a sentence.
- Distance between any two tokens should be consistent (aka linear) across sentences with different lengths.
- Our model should generalize to longer sentences.
- Its values should be bounded (max value).
- It must be deterministic (so no ambiguity).



In other words

- *“We need a function that would allow the model to easily learn to attend by relative positions:*
 - *for any fixed offset k , PE_{pos+k} should be represented as a linear function of PE_{pos} .”*

From: Attention Is All You Need. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, 2017.

<https://arxiv.org/abs/1706.03762>



So, let's use this idea:

Dimension of word
embedding

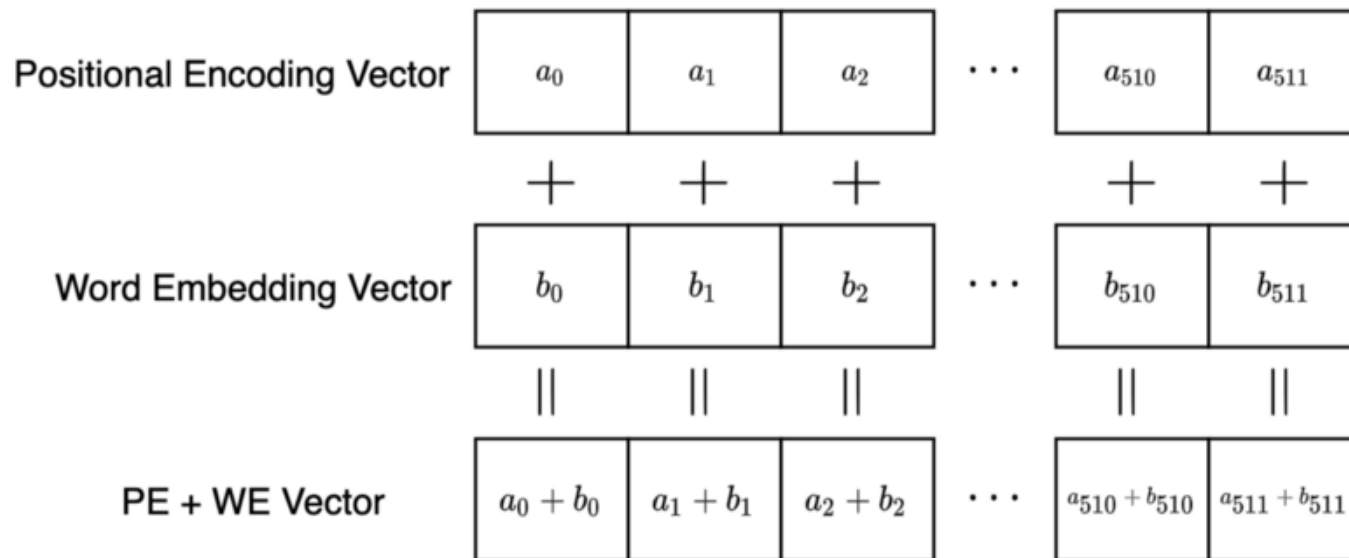


- The encoding proposed by in the original Transformer paper addresses all requirements:
 - It isn't a single number. Instead, it's a d -dimensional vector that contains information about a specific position of the token in a sentence.
 - This encoding is not integrated into the model itself: instead, this vector is used to equip each token with information about its position in a sentence.
 - In other words, we enhance the model's input to inject the order of tokens.

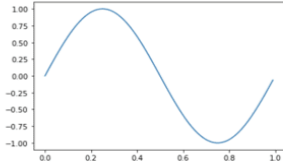
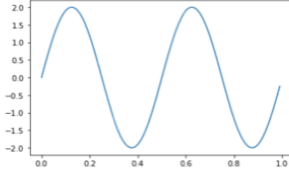
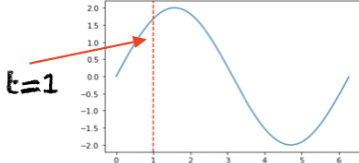
Positional Embeddings

We need to deal with 2 dimensions:

- Token (Word) position (say 8 words in sequence)
- Dimension of embedding for that word (512)
- Idea: if we add a unique value for each position/dimension, we create a unique encoding for each position. Next, we can quantify the similarity between different positions, hence enabling us to encode the relative positions of the tokens (words).
- So, the input and the output of the positional encoding process are matrices.

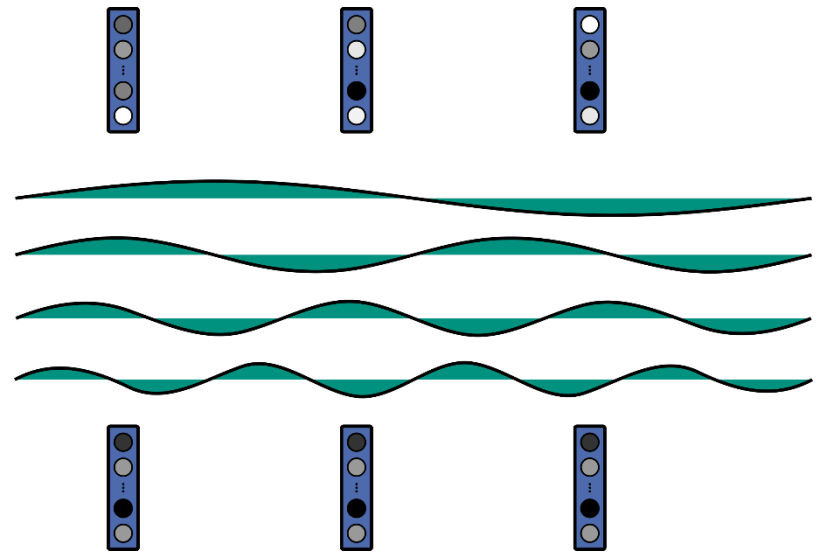


Remember from you Trigonometry

Equation	Graph	Frequency	Wavelength
$\sin(2\pi t)$		1	1
$\sin(2 * 2\pi t)$		2	1/2
$\sin(t)$		$1/2\pi$	2π
$\sin(ct)$	Depends on c	$c/2\pi$	$2\pi/c$

Proposed Positional encoder

- Solution: encode position already in the word embedding:
 - Positional encoding: Idea to add trigonometric function with different wav length to the embedding of each token (word)



Sine and Cosine for the Odd and Even Dimensions

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

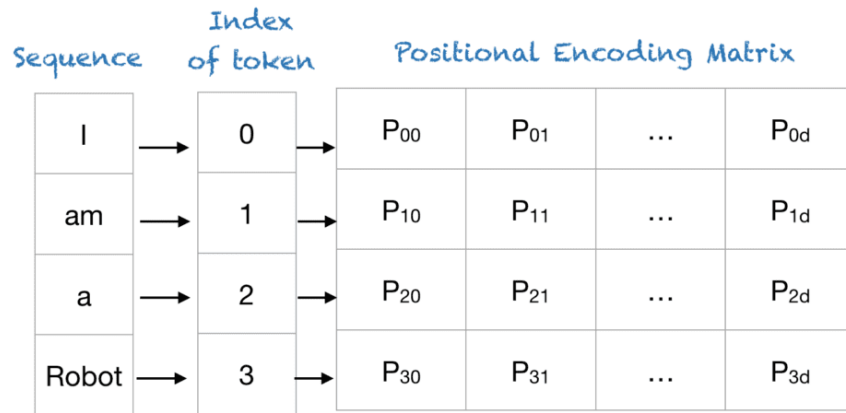
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Why
/2?



- pos*: Position of an object in the input sequence, $0 \leq k < L/2$
- d*: Dimension of the output embedding space (typically 512 or 768)
- PE(pos,i)*: Position function for mapping a position *pos* in the input sequence to index (*pos,i*) of the positional matrix
- n*: User-defined scalar, set to 10,000 by the authors of [Attention Is All You Need](#).
- 2i and 2i+1*: Used for distinguishing sine (even positions) and cosine (odd positions) functions.

Positional Encoding



Positional Encoding Matrix for the sequence 'I am a robot'

Suppose you have an input sequence of length n and require the position of the object within this sequence.

The positional encoding is given by sine and cosine functions of varying frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

Diagram illustrating the calculation of positional encoding values for the sequence "I am a Robot". The sequence is mapped to indices 0, 1, 2, and 3. These indices are used to look up values in a matrix where rows represent the index of the token and columns represent the dimension (0 to 3). The values are calculated using sine and cosine functions of varying frequencies.

Sequence	Index of token, k	Positional Encoding Matrix with $d=4, n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0) = 0$	$P_{01}=\cos(0) = 1$	$P_{02}=\sin(0) = 0$	$P_{03}=\cos(0) = 1$
am	1	$P_{10}=\sin(1/1) = 0.84$	$P_{11}=\cos(1/1) = 0.54$	$P_{12}=\sin(1/10) = 0.10$	$P_{13}=\cos(1/10) = 1.0$
a	2	$P_{20}=\sin(2/1) = 0.91$	$P_{21}=\cos(2/1) = -0.42$	$P_{22}=\sin(2/10) = 0.20$	$P_{23}=\cos(2/10) = 0.98$
Robot	3	$P_{30}=\sin(3/1) = 0.14$	$P_{31}=\cos(3/1) = -0.99$	$P_{32}=\sin(3/10) = 0.30$	$P_{33}=\cos(3/10) = 0.96$

Positional Encoding Matrix for the sequence 'I am a robot'

Positional Embeddings: sin and cos

- In original paper *Attention is all you need* it is stated: “We use sine and cosine functions of different frequencies:

Word / Token Position (0-
(sentence length-1))

dimension from the
Word Embedding (512)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.”

Combining Word Position and Embedding Dim

- Dimension embedding is 0-511
- Token (Word) sequence is max from 0-127
- Since we have 512 dimensions, we have 256 pairs of sine and cosine values. As such, the value of i goes from 0 to 255.

$$\frac{\text{pos}}{10000^{\frac{2i}{512}}}$$

We can then calculate each element of the positional encoding vector for a position (pos) as follows:



$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d_{\text{model}}})$$

$$PE(\text{pos}, 0) = \sin\left(\frac{\text{pos}}{10000^{\frac{0}{512}}}\right)$$

$$PE(\text{pos}, 1) = \cos\left(\frac{\text{pos}}{10000^{\frac{0}{512}}}\right)$$

$$PE(\text{pos}, 2) = \sin\left(\frac{\text{pos}}{10000^{\frac{2}{512}}}\right)$$

$$PE(\text{pos}, 3) = \cos\left(\frac{\text{pos}}{10000^{\frac{2}{512}}}\right)$$

$$PE(\text{pos}, 4) = \sin\left(\frac{\text{pos}}{10000^{\frac{4}{512}}}\right)$$

$$PE(\text{pos}, 5) = \cos\left(\frac{\text{pos}}{10000^{\frac{4}{512}}}\right)$$

⋮

$$PE(\text{pos}, 510) = \sin\left(\frac{\text{pos}}{10000^{\frac{510}{512}}}\right)$$

$$PE(\text{pos}, 511) = \cos\left(\frac{\text{pos}}{10000^{\frac{510}{512}}}\right)$$

Combining it all:

So, the encoding for word position 0 is:

$$\begin{aligned} PE(0) &= \left(\sin\left(\frac{0}{10000^{\frac{0}{512}}}\right), \cos\left(\frac{0}{10000^{\frac{0}{512}}}\right), \sin\left(\frac{0}{10000^{\frac{2}{512}}}\right), \cos\left(\frac{0}{10000^{\frac{2}{512}}}\right), \dots, \sin\left(\frac{0}{10000^{\frac{510}{512}}}\right), \cos\left(\frac{0}{10000^{\frac{510}{512}}}\right) \right) \\ &= (\sin(0), \cos(0), \sin(0), \cos(0), \dots, \sin(0), \cos(0)) \\ &= (0, 1, 0, 1, \dots, 0, 1) \end{aligned}$$

Position 0 is a vector with alternatively repeating 0s and 1s.

Position 1 is as follows:

$$\begin{aligned} PE(1) &= \left(\sin\left(\frac{1}{10000^{\frac{0}{512}}}\right), \cos\left(\frac{1}{10000^{\frac{0}{512}}}\right), \sin\left(\frac{1}{10000^{\frac{2}{512}}}\right), \cos\left(\frac{1}{10000^{\frac{2}{512}}}\right), \dots, \sin\left(\frac{1}{10000^{\frac{510}{512}}}\right), \cos\left(\frac{1}{10000^{\frac{510}{512}}}\right) \right) \\ &= (0.8414, 0.5403, 0.8218, 0.5696, \dots, 0.0001, 0.9999) \end{aligned}$$

The Intuition

- How could this combination of sines and cosines ever represent a position/order?
- Suppose you want to represent a number in binary format, You can do that as follows:

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

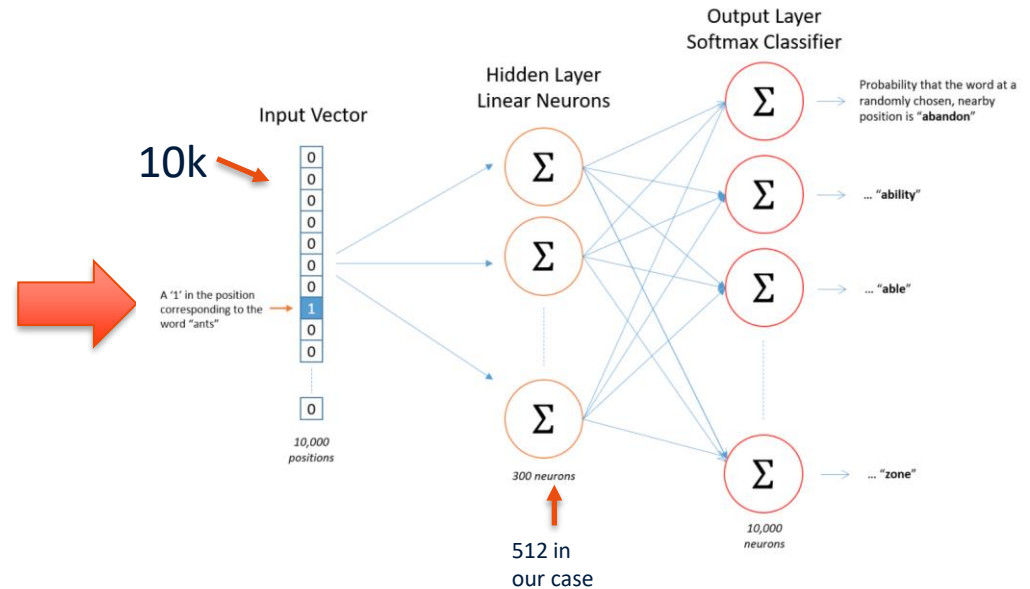


- You can spot the rate of change between different bits. The LSB (Least Significant Bit) bit is alternating on every number, the second-lowest bit is rotating on every two numbers, and so on.
- But using binary values would be a waste of space in the world of floats. So instead, we can use their float continuous counterparts: Sinusoidal functions. These are the equivalent to alternating bits. Moreover, By decreasing their frequencies, we can go from red bits (LSB) to orange ones (Most Significant Bits - MSB). This also works better with the high dimensional word embeddings.

Remember what Word Embeddings look like?

10k

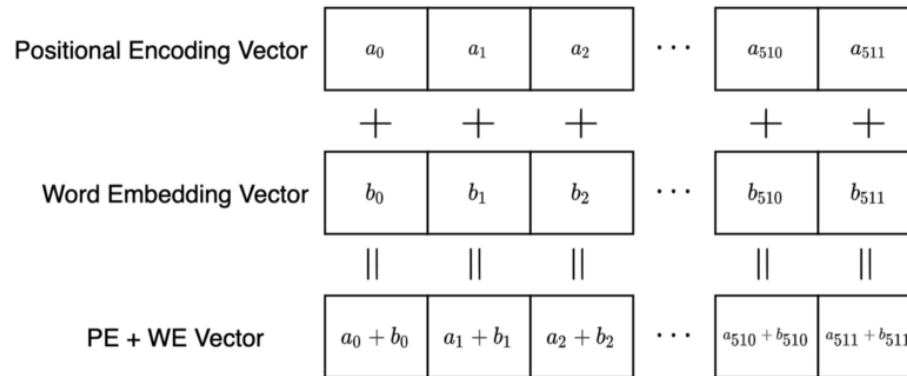
	1	2	3	4	5	6	7	8
I	1	0	0	0	0	0	0	0
ate	0	1	0	0	0	0	0	0
an	0	0	1	0	0	0	0	0
apple	0	0	0	1	0	0	0	0
and	0	0	0	0	1	0	0	0
played	0	0	0	0	0	1	0	0
the	0	0	0	0	0	0	1	0
piano	0	0	0	0	0	0	0	1



- Suppose 10k vocabulary. With one-hot neuron we need 10k matrix.
- But we have only 512 dimensions, so we need to make mapping from 10kx10k to 10k x 512.
- Initialize weights randomly, train hidden layer by using input (word) - output (next word skip grams) values .
- The word representations will get refined and modified throughout training because the weight matrix will get refined and modified throughout training.
- Values for each of the 512 dimensions are normalized between [-1, 1].

Additional Requirements

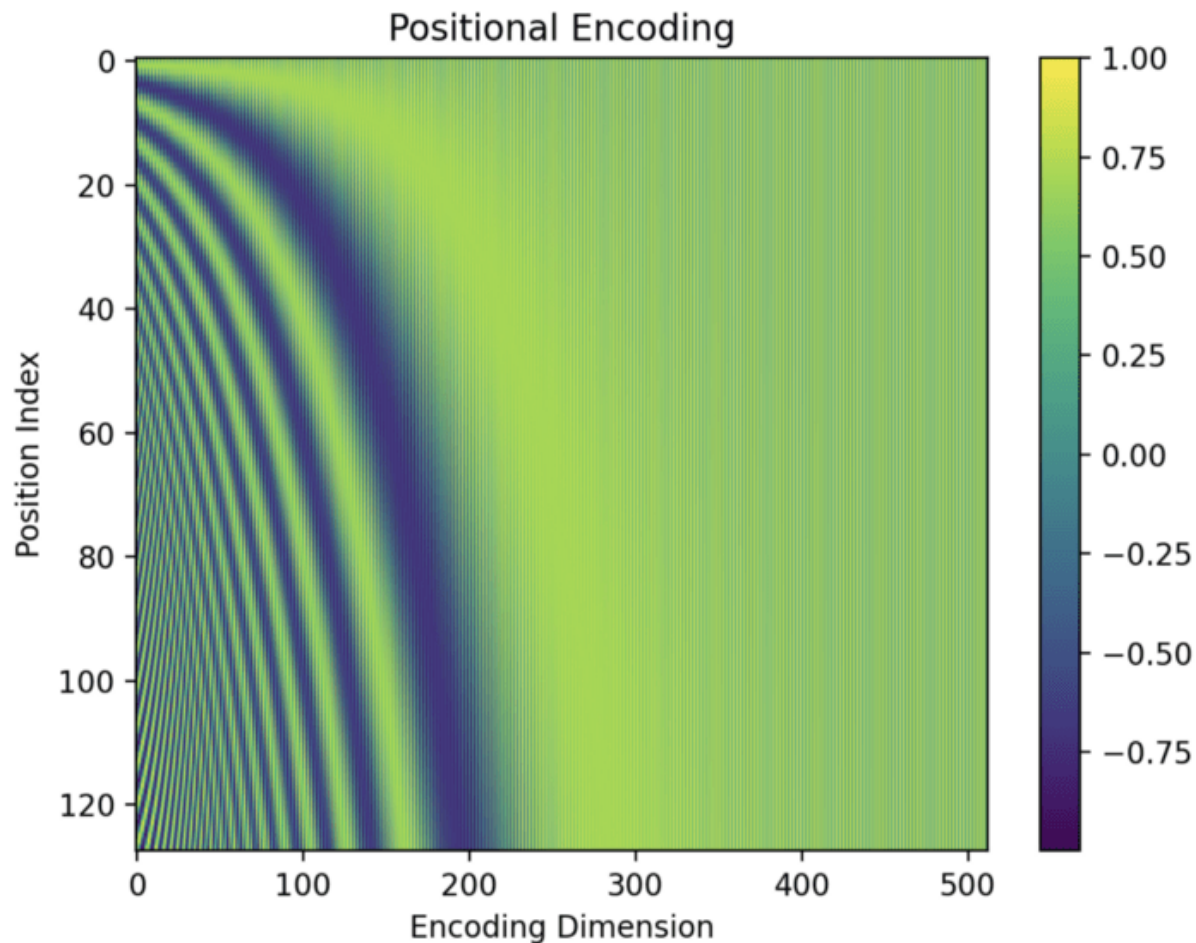
- So, in the original Transformer paper, 512-dimensional word embeddings are used.



- Dimensionality from Word Embeddings need to be the same as that of Positional Encoding.

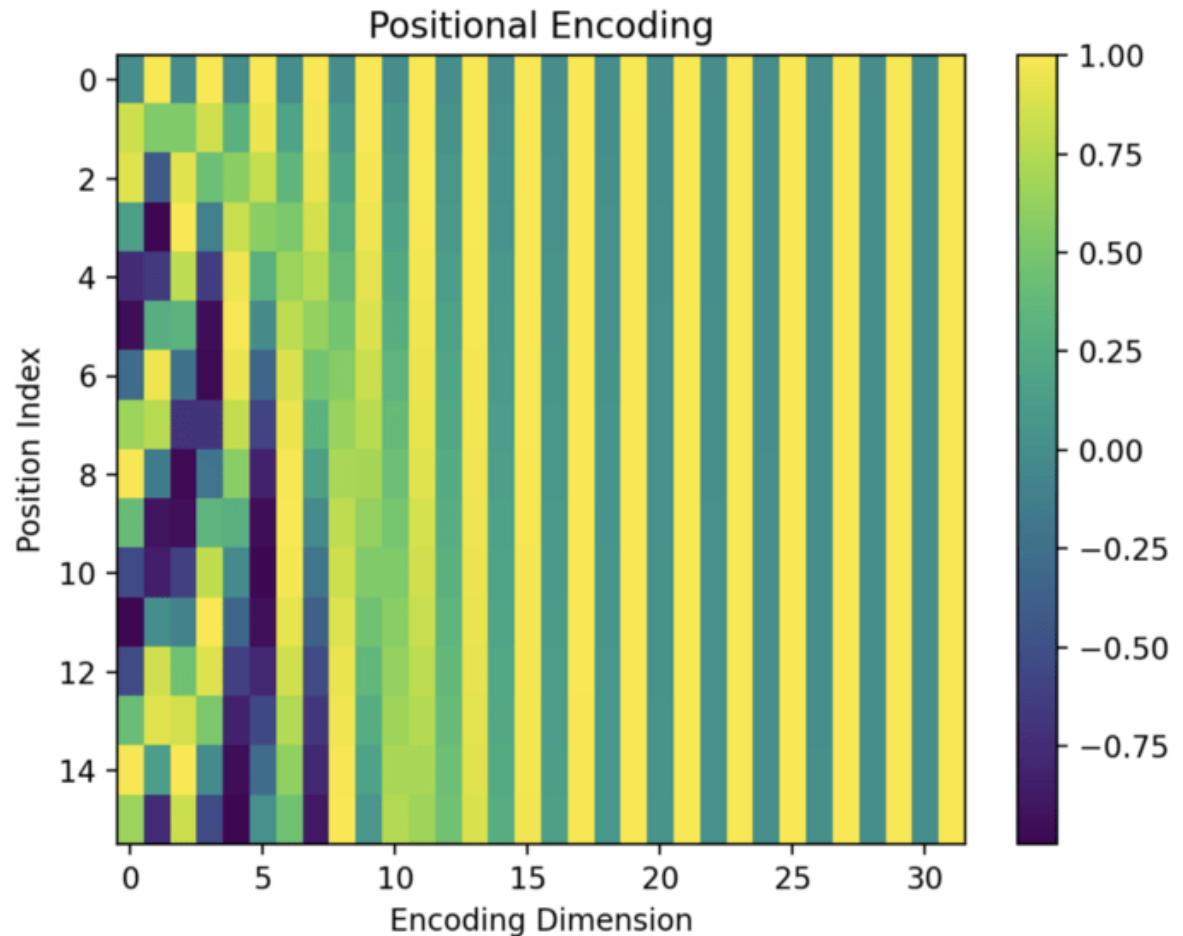
Visualizing the encodings

- For 128 word positions:
The top row is position 0, and the bottom is position 127
- The x-axis is for vector dimension index from 0 to 511.
- The values are between -1 and 1 since they are from sine and cosine functions.
- Darker colors indicate values closer to -1, and brighter (yellow) colors are closer to 1. Green colors indicate values in between. For example, dark green colors indicate values around 0.
- Each row has unique pattern.



Let's look at a less complex model

- 16 word positions
- 32 encoding dimensions
- Each row has a unique pattern.



Additional Slides and Mathematical Proof why Positioning Embeddings work

- Here is a quote from the original paper:

“We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .”

- In other words: if we can find such a transformation, it allows us to represent vector $p_{t+\phi}$ as a linear function of vector p_t for any fixed offset ϕ (aka distance between words).
- This property, makes it easy for the model to learn to attend by relative positions.

Please refer to blog to read the detailed proof <https://timodenk.com/blog/linear-relationships-in-the-transformers-positional-encoding/> .



Well: such a transformation exists:

Let $\mathbf{E} \in \mathbb{R}^{n \times d_{\text{model}}}$ be a matrix that contains d_{model} -dimensional column vectors $\mathbf{E}_{t,:}$, which encode the position t in an input sequence of length n . The function $e : \{1, \dots, n\} \rightarrow \mathbb{R}^{d_{\text{model}}}$ induces this matrix and is defined as

$$e(t) = \mathbf{E}_{t,:} := \begin{bmatrix} \sin\left(\frac{t}{f_1}\right) \\ \cos\left(\frac{t}{f_1}\right) \\ \sin\left(\frac{t}{f_2}\right) \\ \cos\left(\frac{t}{f_2}\right) \\ \vdots \\ \sin\left(\frac{t}{f_{\frac{d_{\text{model}}}{2}}}\right) \\ \cos\left(\frac{t}{f_{\frac{d_{\text{model}}}{2}}}\right) \end{bmatrix}, \quad (1)$$

where the frequencies are

$$f_m = \frac{1}{\lambda_m} := 10000 \frac{2m}{d_{\text{model}}} . \quad (2)$$

The paper states that a linear transformation $\mathbf{T}^{(k)} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ exists, for which

$$\mathbf{T}^{(k)} \mathbf{E}_{t,:} = \mathbf{E}_{t+k,:} \quad (3)$$

holds for any positional offset $k \in \{1, \dots, n\}$ at any valid position $t \in \{1, \dots, n - k\}$ in the sequence.

Derivation - Part 1

That is true, because a definition for $\mathbf{T}^{(k)}$ can be found with no dependence on t :

$$\mathbf{T}^{(k)} = \begin{bmatrix} \Phi_1^{(k)} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \Phi_2^{(k)} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \dots & \Phi_{\frac{d_{\text{model}}}{2}}^{(k)} \end{bmatrix}, \quad (4)$$

where $\mathbf{0}$ denotes 2×2 all-zero matrices and the $\# \frac{d_{\text{model}}}{2}$ transposed rotation matrices $\Phi^{(k)}$ positioned on the main diagonal are defined by

$$\Phi_m^{(k)} = \begin{bmatrix} \cos(r_m k) & -\sin(r_m k) \\ \sin(r_m k) & \cos(r_m k) \end{bmatrix}^T, \quad (5)$$

with wave length r_m (not to be confused with the encoding wave length λ_m).

Now we can reformulate Eq. 3, which we want to prove, as

$$\underbrace{\begin{bmatrix} \cos(r_m k) & \sin(r_m k) \\ -\sin(r_m k) & \cos(r_m k) \end{bmatrix}}_{\Phi_m^{(k)}} \begin{bmatrix} \sin(\lambda_m t) \\ \cos(\lambda_m t) \end{bmatrix} = \begin{bmatrix} \sin(\lambda_m (t + k)) \\ \cos(\lambda_m (t + k)) \end{bmatrix}. \quad (6)$$

Derivation - Part 2

Expanded, that is

$$\sin(\lambda k + \lambda t) = \sin(rk) \cos(\lambda t) + \cos(rk) \sin(\lambda t) \quad (6a)$$

$$\cos(\lambda k + \lambda t) = \cos(rk) \cos(\lambda t) - \sin(rk) \sin(\lambda t), \quad (6b)$$

and we seek to determine r in dependence of λ and k , while eliminating t .

The [addition theorems](#)

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta \quad (7a)$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta \quad (7b)$$

help solving the problem at hand. When applying the addition theorems to the expanded form (Eq. 6a, 6b), it follows

$$\alpha = \lambda k = rk \quad (8a)$$

$$\beta = \lambda t = \lambda t, \quad (8b)$$

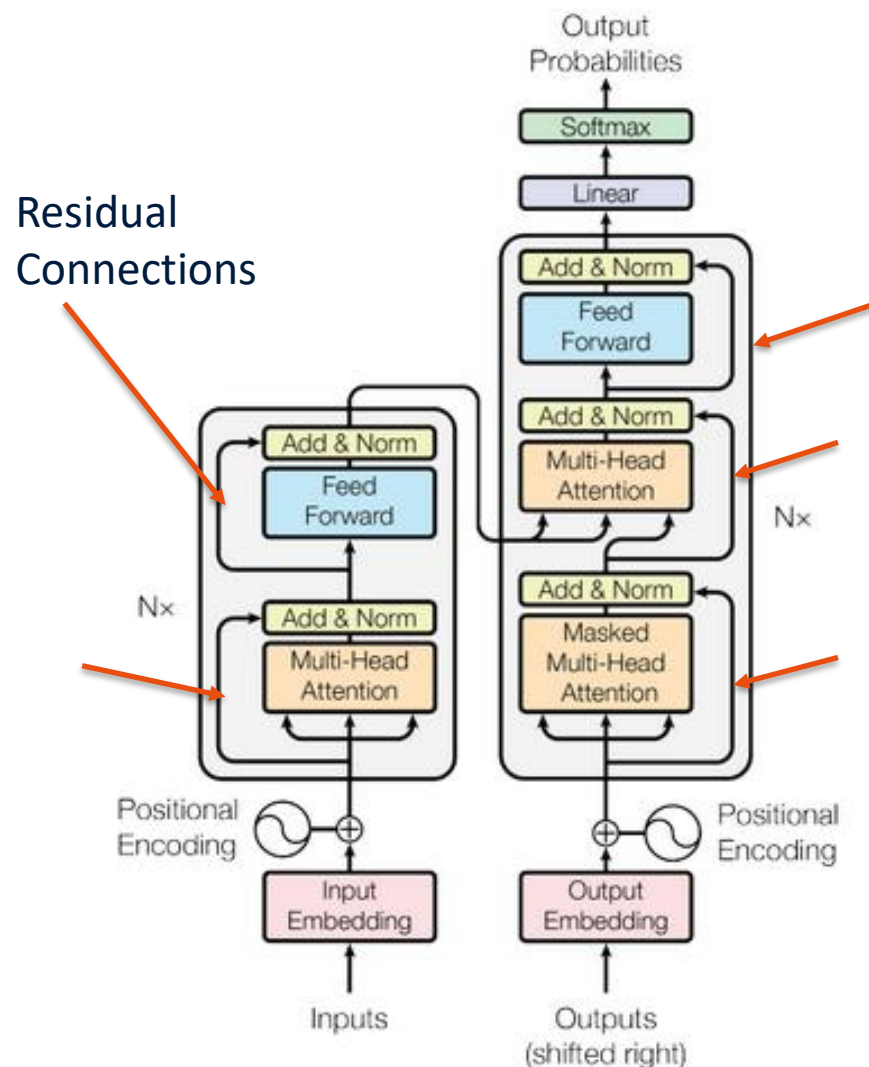
and consequently $r = \lambda$. Applying that finding and the definition of λ_m (Eq. 2) to Eq. 5, we get

$$\Phi_m^{(k)} = \begin{bmatrix} \cos(\lambda_m k) & \sin(\lambda_m k) \\ -\sin(\lambda_m k) & \cos(\lambda_m k) \end{bmatrix}, \quad (9)$$

where $\lambda_m = 10000^{\frac{-2m}{d_{\text{model}}}}$. With that, $\mathbf{T}^{(k)}$ fully specified and depends solely on m , d_{model} , and k . The position within the sequence, t , is not a parameter, q.e.d.

How do we preserve the positional encoding during all layers of N_x ?

- The encoder uses input word embeddings containing positional encodings as discussed and subsequently applies the MH-SA operations.
- Why doesn't the model lose the word positions in the process?
- The transformer avoids this problem by **residual connections**.
- Doing so will carry over the positional encodings through the multiple layers of the encoder and decoder.



Cross Attention and the Decoder

Driving next-word prediction
using the outcome of the
MH-SA calculation



Maastricht University

From the Encoder to the Decoder

What the encoder gives

- After the encoder finishes, each source token x_i is turned into a hidden vector $h_i^{enc} \in \mathbb{R}^d$.
- Collectively these vectors form a **memory**:

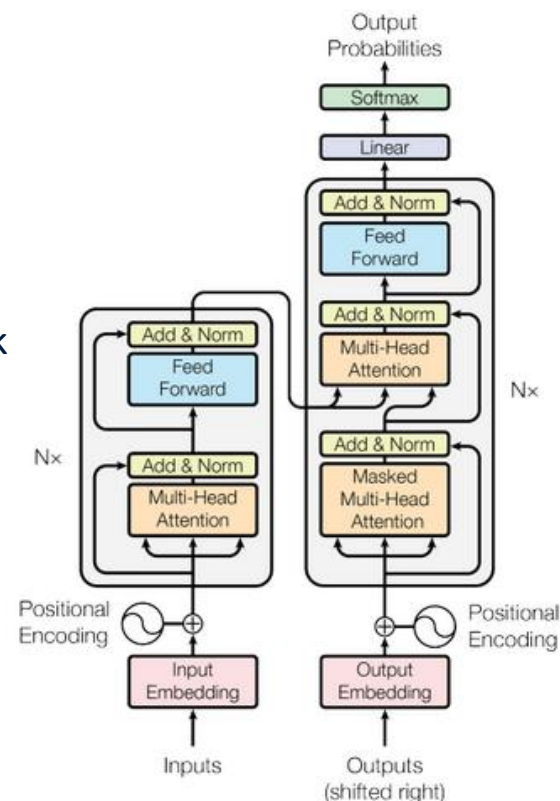
$$H^{enc} = [h_1^{enc}, h_2^{enc}, \dots, h_{n_{src}}^{enc}]$$

Decoder layer structure

Each decoder layer has **two attention blocks**:

- Self-attention (masked)** – lets the decoder token at position t look at **previously generated target tokens** (but not the future).
- Cross-attention** – lets each decoder position attend to the **encoder outputs** H^{enc} .

After these, the layer passes through a feed-forward network.

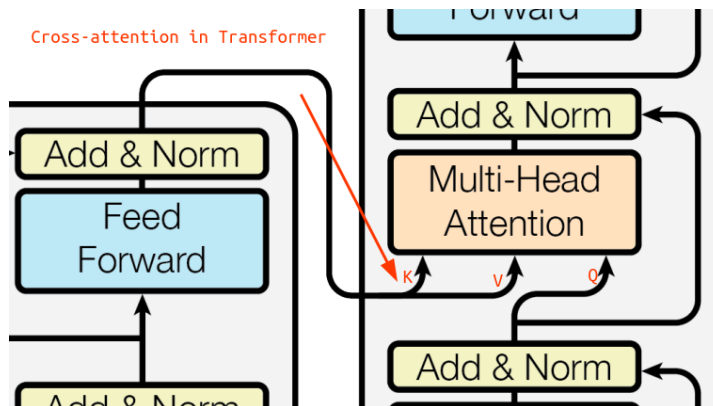


Cross Attention

- Cross-attention in transformers is a mechanism that allows the decoder to access relevant information from the encoder, enhancing tasks like translation by focusing on important details.
- It helps aligning words from the source language with their corresponding words in the target language (or text in case of abstracting) for more accurate outputs.

How Cross-Attention Works

1. *Encoder-Decoder Interaction:*
 - The encoder processes input data (e.g., a sentence) and generates representations.
 - The decoder uses these representations to produce output (e.g., a translated sentence).
2. *Query, Key, and Value:*
 - The decoder creates a "query" for each word it generates.
 - The encoder provides "keys" and "values" that help identify important parts of the input.
3. *Attention Calculation:*
 - The attention mechanism calculates the relevance of each key to the query, allowing the decoder to focus on the most pertinent information.



Cross-Attention Mechanism

For each target position t in the decoder:

1. The decoder's current hidden state q_t (query) is projected from the decoder's previous layer.
2. Keys K and values V come from the **encoder outputs** H^{enc} :

$$K = H^{enc} W^K, V = H^{enc} W^V$$

3. The attention weights are:

$$\alpha_{t,i} = \text{softmax} \left(\frac{q_t K^\top}{\sqrt{d_k}} \right)_i$$

How much target step t attends to source token i).

4. The **context vector** is:

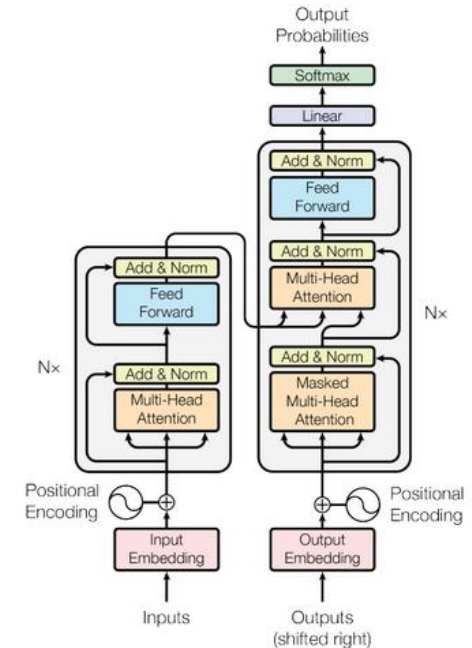
$$c_t = \sum_i \alpha_{t,i} V_i$$

5. This c_t is added to the decoder's hidden state, giving it **source-conditioned information**.



Generation Step

- The decoder now has a hidden state that knows both:
 - **Its own history** (via masked self-attention),
 - **The source sequence** (via cross-attention).
- A linear layer + Softmax converts this hidden state into a probability distribution over the vocabulary:
$$P(y_t, | y_{<t}X) = \text{Softmax}(W_o h_t)$$
- The model samples or chooses the most likely next token, appends it to the sequence, and repeats.



Cross-attention = the decoder “looking back” at the encoder memory to decide what to say next. At each decoding step, the query comes from what’s been generated so far; the keys/values are fixed from the encoded input. This is how the model grounds each generated token in the source sequence.

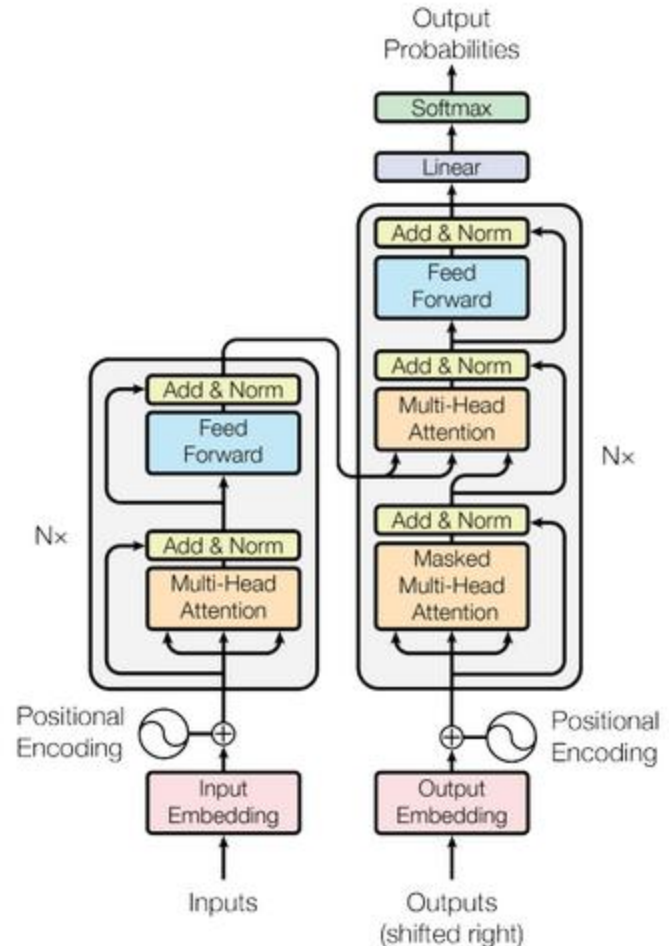
More Parameters, Training and Computational Complexity



Maastricht University

Transformer Parameters

- Input is using 768 dimensional embeddings (BERT Embeddings), one per word. 512 was the dimensionality in the original Transformer paper.
- Sentences are split into a fixed number of segments. Typically, 12-24. But in this case let's say 8.
- Number of layers, N_x is typically 6.
- Heads for multi-headed attention are the same as the number of sentence segments. Also 12-24. But we can also take 8 here.
- Then $768/12 = 64$ (dimensionality per head for BERT embeddings). Or $512 / 8 = 64$ in the original transformer setup
- Other dimension is the number of words in our window that we observe.



Computational Complexity of Transformers

Each Transformer layer has:

1. Self-attention

- Query, Key, Value projections: $O(n \cdot d^2)$
- Dot product attention: $O(n^2 \cdot d)$ (because every token attends to every other token)
- Output projection: $O(n \cdot d^2)$

2. Feed-forward network (FFN)

- Two dense layers with hidden size d_{ff}
(usually $\sim 4d$): $O(n \cdot d \cdot d_{ff})$



Here:

- n = sequence length (context size)
- d = hidden dimension
- $d_{ff} \approx 4d$

Per-layer cost summary:

$$O(n^2 d) + O(nd^2) + O(ndd_{ff})$$

Since $d_{ff} \sim 4d$, the **dominant terms** are usually:

$$\text{Self-Attention: } O(n^2 d) \quad \text{FFN: } O(nd^2)$$

For **long sequences**, $n^2 d$ dominates (attention becomes the bottleneck).

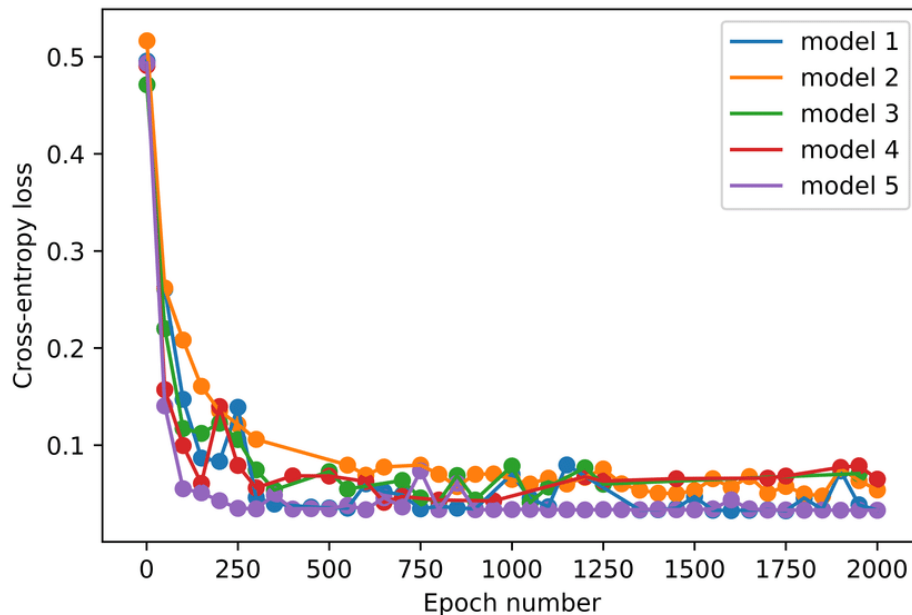
For **shorter sequences but large model width**, nd^2 (FFN) can dominate.

Memory complexity:

- Attention weights: $O(n^2)$
- Activations (for backprop): $O(nd)$ per layer
- Total per layer: roughly $O(n^2 + nd)$

Training Transformers

- Annotated training data: e.g. English – German sentence pairs.
- Feed English into encoder. German as result on decoder.
- Minimize the loss function as in back propagation (but now via decoder-encoder and all layers in between).
- Transformers use Cross Entropy:



$$E = \sum_x \sum_{i=1}^n t_i^x * \ln(o_i^x)$$

There exist many optimizations and you need a lot of data points!

In the lecture “Fine Tuning Large-Language Models”, we will take a close look at training and fine tuning.

Computational Complexity Training Transformers

Increasing context length (n) hurts more than increasing model width (d).

Because attention scales as n^2 , FFN as d^2

This is why long context windows explode GPU memory: activations depend on n and d not on model size d

Inference (single forward pass):

$$O(n^2d + nd^2)$$

Training (forward + backward + update):

$$O(3(n^2d + nd^2))$$

Total training cost over dataset of size D and L layers:

$$6DL(n^2d + nd^2)$$

Memory:

$$O(ndL + n^2)$$

The Transformer Zoo of (Large) Language Models and Generative AI



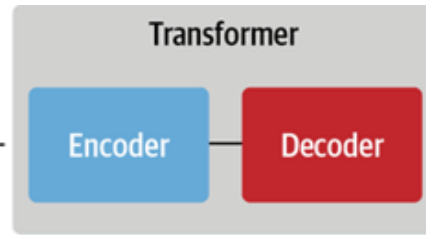
Maastricht University

The Transformer Zoo

Stanford
Alpaca



TensorFlow

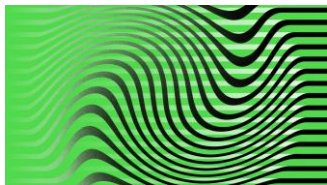


Gemini

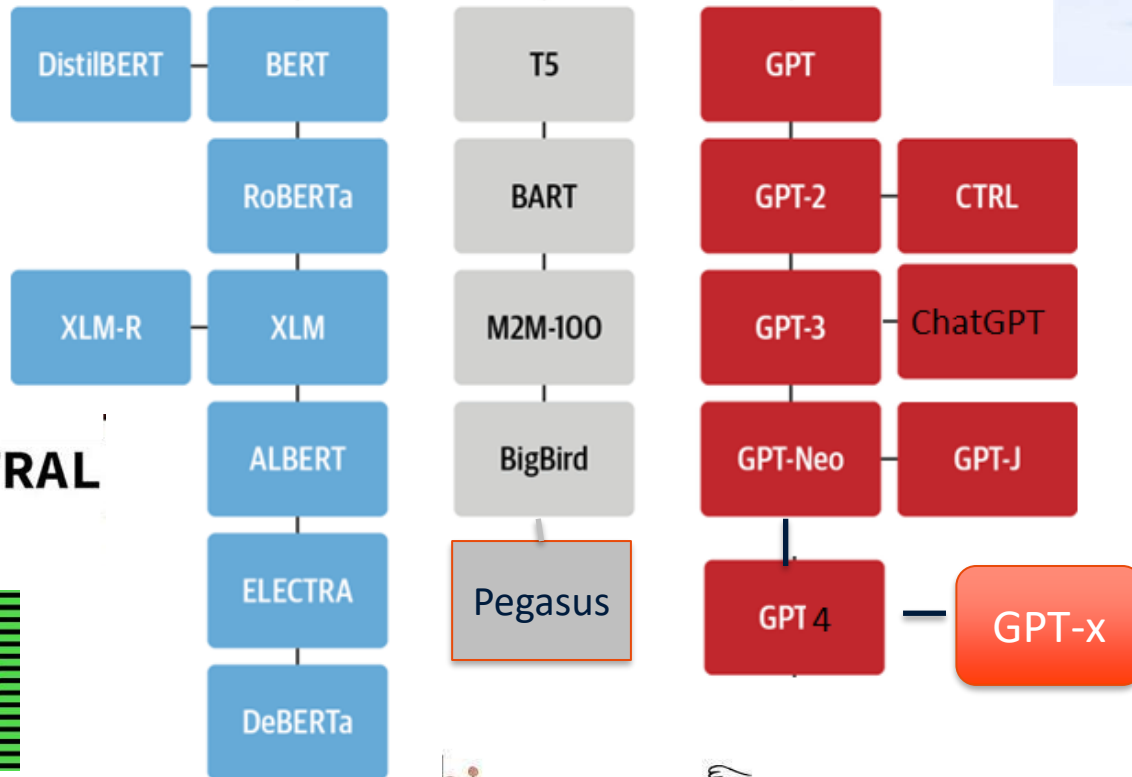


PyTorch

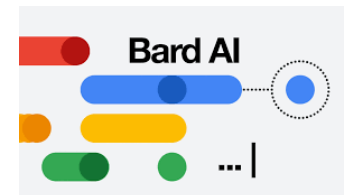
MISTRAL
AI_



Maastricht University



Meta



Next Steps

- In this lecture we discussed encoder-decoder transformers.
- In the subsequent lectures we will discuss encoder-only and decoder-only models in more details.

Questions ...



Maastricht University

Exercise 1: Exercise 1: Scaled Dot-Product Mechanics and Multi-Head Dimensions

In the Transformer architecture (*Attention Is All You Need*), multi-head self-attention linearly projects input representations into multiple representation subspaces to compute attention in parallel.

Assume an input sequence of length $n = 16$ tokens processed with the standard base configuration: model embedding dimension $d_{\text{model}} = 512$ and $h = 8$ attention heads.

- Determine the dimensionality per head (d_k and d_v). For a single attention head i , state the exact matrix dimensions of the query projection weight W_i^Q , the query matrix Q , the key matrix K , and the transposed key matrix K^T .
- Write down the scaled dot-product attention formula:
- $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$
- What is the matrix shape of the raw attention logits QK^T ? Along which axis is the softmax applied, and why is scaling by $\frac{1}{\sqrt{d_k}}$ necessary when computing dot products in high dimensions?
- Show the dimensions of the single-head output matrix $Z_i = \text{Attention}(Q_i, K_i, V_i)$. Once all $h = 8$ heads are concatenated ($\text{Concat}(\text{head}_1, \dots, \text{head}_h)$), what is the resulting dimension, and what are the dimensions of the final linear projection matrix W^O needed to project the multi-head output back to d_{model} ?

Exercise 2: Sinusoidal Positional Encoding Derivation and Properties

Because Transformers process all tokens in parallel rather than sequentially through recurrent loops, the architecture lacks inherent awareness of token order. To inject sequence information, fixed sinusoidal positional encodings (PE) are added element-wise to the input token embeddings.

- $PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$
- Calculate the exact values for the first four dimensions ($2i = 0, 2i + 1 = 1, 2i = 2, 2i + 1 = 3$) for token position $pos = 0$. What repeating numerical pattern appears across the entire vector for $pos = 0$?
- The paper hypothesizes that this formulation enables the model to easily learn relative positions because for any fixed offset k , PE_{pos+k} can be represented as a linear transformation of PE_{pos} . Using the trigonometric addition theorems for $\sin(\alpha + \beta)$ and $\cos(\alpha + \beta)$, describe how a 2×2 rotation matrix $\Phi_m^{(k)}$ maps the pair $(\sin(\lambda_m \cdot pos), \cos(\lambda_m \cdot pos))$ to $(\sin(\lambda_m(pos + k)), \cos(\lambda_m(pos + k)))$ independently of the absolute position pos .
- Why are residual connections ($x + \text{Sublayer}(x)$) essential after each attention block to ensure that these positional signals are not lost or overwritten as representations pass through deep stacks ($N \times$) of encoder and decoder layers?

Exercise 3: Encoder-Decoder Cross-Attention and Computational Bottlenecks

Review the structural flow of the complete Transformer encoder-decoder model and its computational complexity.

- In the decoder block, distinguish between **Masked Multi-Head Attention** and **Encoder-Decoder Cross-Attention**:
 - Why is causal masking required in the decoder's self-attention during autoregressive training?
 - In cross-attention, which sublayer provides the queries (Q), and which sublayer provides the keys (K) and values (V)? Explain how this routes information from the source sentence to guide target next-word generation.
- Explain the structural purpose of the position-wise Feed-Forward Network (FFN) placed after attention in each layer. Why would stacking multiple self-attention layers without non-linear feed-forward layers collapse into a series of linear re-averaging operations?
- Consider the computational complexity of a Transformer layer: self-attention scales as $O(n^2d)$, while the feed-forward network scales as $O(nd^2)$ (where n is sequence length and d is model dimension). When scaling models to long document contexts (e.g., $n > 8,000$), explain why the self-attention operation—and its corresponding attention weight memory $O(n^2)$ —becomes the primary computational and memory bottleneck on modern hardware.

Discussion



On Hardware vs. Linguistic Design: Planned Breakthrough or Hardware Lottery?

In 2017, the authors of 'Attention Is All You Need' discarded recurrence entirely, demonstrating that self-attention—built purely from matrix multiplications (QK^TV)—could outperform complex recurrent networks like LSTMs.

Yet Google's TPUs and modern GPUs were designed specifically to excel at parallel tensor dot products.

Did the Transformer triumph because non-recurrent self-attention is truly the optimal mathematical structure for human language, or did it win because it was the first NLP architecture designed to match what parallel silicon chips compute fastest?

If hardware architecture dictates algorithmic dominance, are we discovering true linguistic computation, or merely whatever fits on a GPU?

On Positional Encodings: The Paradox of an Orderless Architecture

A foundational premise of language is that sequential order governs semantics: 'Jane hits John' conveys an entirely different event than 'John hits Jane'.

However, because self-attention compares all tokens to all other tokens simultaneously in parallel, the Transformer is fundamentally permutation-invariant—without positional vectors, it treats text as an orderless bag of words. Vaswani et al. patched this by adding sinusoidal waves of varying frequencies directly into the input embeddings.

If an architecture possesses no intrinsic temporal awareness and requires external mathematical scaffolding to distinguish word order, does it actually process language, or is it an orderless spatial calculator forced to simulate sequential time?

On the Quadratic Wall: The Geometric Cost of 'Everything-to-Everything'

Self-attention achieves expressive power by computing full pairwise dot products across the entire sequence, comparing every token to every other token. This produces an attention matrix of size $n \times n$, resulting in a quadratic computational and memory complexity of $O(n^2)$.

When humans read a 500-page book, we do not continuously recompute cross-attention between every single word and all preceding tokens across hundreds of pages.

If biological intelligence processes unbounded language using bounded, local cognitive bandwidth, is the Transformer's $O(n^2)$ 'compare everything to everything' brute-force mechanism a dead-end for scaling long-context AI?

Why can't we achieve global semantic understanding without computing the entire quadratic attention matrix?

References on Transformers & BERT

- <https://nlp.seas.harvard.edu/2018/04/03/attention.html>
- A. Vaswani et al., “Attention Is All You Need”, (2017). This title was so catchy that no less than 50 follow-up papers have included “all you need” in their titles!
- J. Howard and S. Ruder, “Universal Language Model Fine-Tuning for Text Classification”, (2018).
- A. Radford et al., “Improving Language Understanding by Generative Pre-Training”, (2018).
- J. Devlin et al., “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding”, (2018).
- I. Sutskever, O. Vinyals, and Q.V. Le, “Sequence to Sequence Learning with Neural Networks”, (2014).
- D. Bahdanau, K. Cho, and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate”, (2014).
- A. Radford, R. Jozefowicz, and I. Sutskever, “Learning to Generate Reviews and Discovering Sentiment”, (2017).
- Y. Zhu et al., “Aligning Books and Movies: Towards Story-Like Visual Explanations by Watching Movies and Reading Books”, (2015).

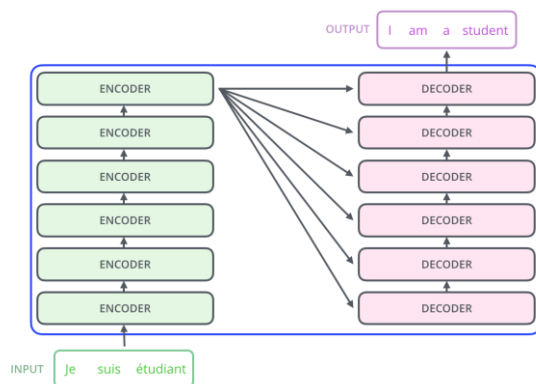
>>> Should you decide to read only 2 papers in this course, then make sure it will be these two!



BERT: Attention, Explanation & Understanding

- [1]: Attention is not Explanation, Sarthak Jain, Byron C. Wallace. <https://arxiv.org/abs/1902.10186>.
- [2]: Attention is not not Explanation. Sarah Wiegrefe and Yuval Pinter.
<https://arxiv.org/abs/1902.10186>.
- [3]: Is Attention Interpretable? Sofia Serrano and Noah A. Smith. <https://arxiv.org/abs/1906.03731>.
- [4]: Timothy Niven and Hung-Yu Kao. 2019. Probing neural network comprehension of natural language arguments. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pages 4658–4664, Florence, Italy. Association for Computational Linguistics (on Clever Hans effect). <https://www.aclweb.org/anthology/P19-1459/>
- [5]: NLP’s Clever Hans Moment has Arrived. Benjamin Heinzerling. August 26, 2019.
<https://thegradient.pub/nlps-clever-hans-moment-has-arrived/>.

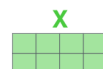
Good (Visual) Tutorials on Transformers & BERT



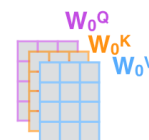
1) This is our input sentence*

Thinking Machines

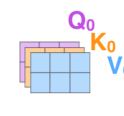
2) We embed each word*



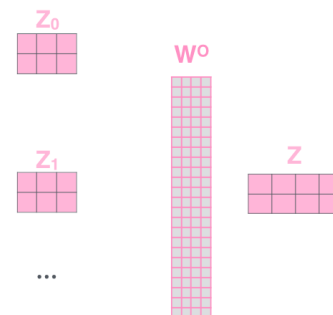
3) Split into 8 heads. We multiply X or R with weight matrices



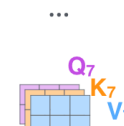
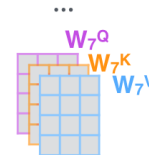
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



- <https://jalammar.github.io/illustrated-transformer/>
- <https://jalammar.github.io/a-visual-guide-to-using-bert-for-the-first-time/>